ใบอนุมัติโครงงานปริญญานิพนธ์
สาขาวิชาเทคโนโลยีสารสนเทศ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญานิพนธ์ ระบบตรวจจับและวิเคราะห์อารมณ์จากใบหน้าในคลิปสั้นโดยใช้การเรียนรู้เชิงลึก
Emotion Recognition and Analysis System for Facial Expressions in Short Clips Using Deep Learning Model

ผู้จัดทำ นางสาว ณัฐลดา กตปุญญกุล

ปีการศึกษา 2567

โครงงานปริญญานิพนธ์นี้ได้รับการอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาเทคโนโลยีสารสนเทศ
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.รัชดาพร คณาวงษ์)

..... กรรมการสอบ
(ผู้ช่วยศาสตราจารย์ ดร.สิรักข์ แก้วจำนงค์)

..... อาจารย์ที่ปรึกษาและกรรมการสอบ
(อาจารย์ ดร.ปัญญนัท อ้นพงษ์)

ระบบตรวจจับและวิเคราะห์อารมณ์จากใบหน้าในคลิปสั้นโดยใช้การเรียนรู้เชิงลึก

Emotion Recognition and Analysis System for Facial Expressions in Short Clips Using Deep Learning Model

ณัฐลดา กตปุญญกุล

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาเทคโนโลยีสารสนเทศ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร
ปีการศึกษา 2567

ชื่อปริญญานิพนธ์	ระบบตรวจจับและวิเคราะห์อารมณ์จากใบหน้าในคลิปสั้นโดยใช้การเรียนรู้เชิงลึก Emotion Recognition and Analysis System for Facial Expressions in Short Clips Using Deep Learning Model
ผู้จัดทำ	นางสาว ณัฐลดา กตปุญญกุล
อาจารย์ที่ปรึกษา	อาจารย์ ดร.ปัญญนัท อ้นพงษ์
ชื่อปริญญา	วิทยาศาสตรบัณฑิต (สาขาเทคโนโลยีสารสนเทศ)
ปีการศึกษา	2567

บทคัดย่อ

โครงการปริญญานิพนธ์มุ่งเน้นการวิเคราะห์อารมณ์ของผู้ชมต่อคลิปวิดีโอ เพื่อช่วยให้ผู้ใช้เข้าใจอารมณ์ที่แท้จริงของตนขณะรับชม การศึกษาเรื่องนี้มีความสำคัญเนื่องจากปฏิกิริยาทางอารมณ์ของผู้ชมอาจส่งผลต่อการประเมินเนื้อหาที่ไม่ชัดเจนได้ ทางผู้วิจัยจึงได้ใช้เทคโนโลยีการเรียนรู้ของเครื่อง (Machine Learning) และโครงข่ายประสาทเทียม (Artificial Neural Networks) เพื่อทำการตรวจจับและวิเคราะห์ใบหน้า โดยเฉพาะการแสดงออกทางสีหน้า ซึ่งจะช่วยระบุอารมณ์ของผู้ใช้ในแต่ละช่วงของคลิปวิดีโอ วัตถุประสงค์หลักของโครงการนี้คือการเข้าใจว่าผู้ชมตอบสนองทางอารมณ์ต่อเนื้อหาประเภทต่าง ๆ อย่างไร และการทดสอบฉากหรือเนื้อหานำร่องเพื่อตรวจสอบและปรับปรุงเนื้อหาให้เหมาะสมกับผู้ชม ขอบเขตของการวิจัยจะเน้นไปที่การติดตามใบหน้าและการวิเคราะห์การแสดงออกทางสีหน้าเพื่อวัดปฏิกิริยาทางอารมณ์ ผลการดำเนินงานแสดงให้เห็นว่าการวิเคราะห์อารมณ์จากการแสดงออกทางสีหน้าในระหว่างการรับชมวิดีโอสามารถช่วยปรับปรุงประสบการณ์การใช้งานและคุณภาพของเนื้อหาที่นำเสนอได้ ข้อเสนอแนะจากการศึกษานี้ชี้ให้เห็นถึงโอกาสในการพัฒนาเทคโนโลยีสื่อที่มีความเป็นมิตรกับผู้ใช้มากยิ่งขึ้นในอนาคต

คำสำคัญ: การแสดงออกทางสีหน้า, การวิเคราะห์ใบหน้า

Keyword: Facial Expressions, Facial Analysis

กิตติกรรมประกาศ

โครงงานปริญญานิพนธ์เรื่องระบบตรวจจับและวิเคราะห์อารมณ์จากใบหน้าในคลิปสั้นโดยใช้การเรียนรู้เชิงลึก ปริญญานิพนธ์นี้สำเร็จลุล่วงไปได้ด้วยความกรุณาและการสนับสนุนจากหลายฝ่าย

ขอขอบพระคุณอาจารย์ที่ปรึกษา อาจารย์ ดร.ปัญญนัท อ้นพงษ์ ที่ได้ให้คำแนะนำที่เป็นประโยชน์อย่างยิ่งตลอดกระบวนการทำวิจัย ทั้งในด้านแนวคิด ความรู้ และกำลังใจ ขอบคุณสำหรับการสนับสนุนทางวิชาการและการชี้แนะแนวทางในการแก้ไขปัญหาและหพัฒนาปริญญานิพนธ์นี้จนสำเร็จ

ขอขอบคุณ คณะกรรมการสอบ ทุกท่านที่ได้สละเวลาในการให้ข้อเสนอแนะที่สำคัญเพื่อปรับปรุงงานวิจัยนี้ให้มีความสมบูรณ์มากยิ่งขึ้น นอกจากนี้ ขอบคุณ เพื่อนร่วมโครงงานปริญญานิพนธ์ ที่ร่วมแรงร่วมใจกันในการพัฒนาระบบและทำงานร่วมกันอย่างเต็มที่

สุดท้ายนี้ ขอขอบคุณ ครอบครัวและเพื่อน ๆ ที่ให้กำลังใจและสนับสนุนในการทำโครงงานนี้จนสำเร็จลุล่วงด้วยดี

ณัฐลดา กตปุญญกุล

สารบัญ

หน้า

บทคัดย่อ ..... ก
กิตติกรรมประกาศ ..... ข
สารบัญ ..... ค
สารบัญตาราง ..... จ
สารบัญรูป ..... ฉ
บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ 1
1.3 ลักษณะและขอบเขต 1
1.4 อุปกรณ์และเครื่องมือที่ใช้ 1
1.4.1 ฮาร์ดแวร์ 1
1.4.2 ซอฟต์แวร์ 1
1.5 ผลที่คาดว่าจะได้รับ 2
บทที่ 2 ผลงานที่เกี่ยวข้อง 3
2.1 Findface 3
2.2 EmoVu 4
2.3 iMotions 5
2.4 SkyBiometry 6
2.5 Microsoft Project Oxford 7
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง 8
3.1 Convolutional Neural Networks (CNN) 8
3.2 machine learning (ML) 9
3.3 Artificial Neural Network (ANN) 10
3.4 Recurrent Neural Network (RNN) 11
3.5 Image Classification 11
3.6 Image Detection 12
บทที่ 4 ขั้นตอนและแผนการดำเนินงาน 13
4.1 ขั้นตอนการดำเนินงาน 13
4.1.1 การคิดหัวข้อโครงงานและรวบรวมข้อมูล 13
4.1.2 กำหนดขอบเขตและวางแผนการดำเนินงาน 13
4.1.3 ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง 13
4.1.4 ออกแบบระบบงาน 13
4.1.5 การพัฒนาระบบ 13

4.1.6 การทดสอบระบบ 13
4.1.7 จัดทำเอกสารประกอบโครงงาน 13
4.2 แผนการดำเนินงาน 14
บทที่ 5 การวิเคราะห์และออกแบบระบบ 15
5.1 ภาพรวมของที่มาของโค้ดและเทคนิคการนำมาใช้ในระบบ 15
5.1.1 ที่มาของโค้ดและเทคนิคการนำมาใช้ในระบบ 15
5.2 ภาพรวมการทำงานและการทำงานของระบบ 18
5.2.1 โครงสร้างของระบบ 18
5.2.2 รายละเอียดการทำงานของระบบ 18
5.2.3 ชุดคำสั่งและฟังก์ชันการทำงานของระบบ 19
5.3 กระบวนการทำงานรูปแบบแผนผัง Block Diagram 30
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ 32
6.1 ผลการดำเนินงาน 32
6.2 ข้อจำกัด 36
6.3 ข้อเสนอแนะ 36
บรรณานุกรม 37
ภาคผนวก ก ความหมายของกราฟ 38
ภาคผนวก ข การติดตั้งโปรแกรม 39

สารบัญตาราง

หน้า

ตารางที่ 1 แผนการดำเนินงาน 14

สารบัญรูป

รูปที่ 2.1 Findface 3
รูปที่ 2.2 Emovu 4
รูปที่ 2.3 iMotions 5
รูปที่ 2.4 SkyBiometry 6
รูปที่ 2.5 SkyBiometry 7
รูปที่ 3.1 Convolutional Neural Networks 8
รูปที่ 3.2 machine learning 9
รูปที่ 3.3 Artificial Neural Network 10
รูปที่ 3.4 Recurrent Neural Network 11
รูปที่ 3.5 Image Classification 12
รูปที่ 3.6 Image Detection 12
รูปที่ 5.1 การใช้ Machine Learning Model 15
รูปที่ 5.2 โมเดลที่นำมาใช้งาน 15
รูปที่ 5.3 การข้ามเฟรม (Frame skip) 16
รูปที่ 5.4 การนำไปประยุกต์ใช้โค้ดและขั้นตอนในระบบการทำงาน 17
รูปที่ 5.5 ช่วงเวลาและลำดับการแสดงอารมณ์ 17
รูปที่ 5.6 คำสั่ง Library ที่นำมาใช้ทั้งหมด 19
รูปที่ 5.7 คลาส VideoPlayer 19
รูปที่ 5.8 คลาส VideoPlayer 20
รูปที่ 5.9 ฟังก์ชัน load_video 20
รูปที่ 5.10 ฟังก์ชัน start_video_and_recording 21
รูปที่ 5.11 ฟังก์ชัน play_video 21
รูปที่ 5.12 ฟังก์ชัน record_screen_and_cam 22
รูปที่ 5.13 ฟังก์ชัน stop_recording 23
รูปที่ 5.14 ฟังก์ชัน analyze_emotions 23
รูปที่ 5.15 ฟังก์ชัน detect_emotions 24
รูปที่ 5.16 ฟังก์ชัน create_combined_image 25
รูปที่ 5.17 ฟังก์ชัน process_video (1) 26
รูปที่ 5.18 ฟังก์ชัน process_video (2) 26
รูปที่ 5.19 ฟังก์ชัน process_video (3) 27
รูปที่ 5.20 ฟังก์ชัน plot_emotion_distribution (1) 27
รูปที่ 5.21 ฟังก์ชัน plot_emotion_distribution (2) 28
รูปที่ 5.22 ฟังก์ชัน plot_emotion_distribution (3) 28
รูปที่ 5.23 ฟังก์ชัน plot_emotion_distribution (4) 29

รูปที่ 6.1 โปรแกรม Video Player with Emotion Analysis 32
รูปที่ 6.2 โฟลเดอร์ที่เก็บวิดีโอ 33
รูปที่ 6.3 แสดงวิดีโอที่เลือก 33
รูปที่ 6.4 เลือกวิดีโอที่อัดหน้าจอของผู้ใช้ 34
รูปที่ 6.5 การประมวผลวิดีโอ 34
รูปที่ 6.6 กราฟวงกลม (Pie Chart) 34
รูปที่ 6.7 กราฟแท่ง (Bar Chart) 35
รูปที่ 6.8 ผลความน่าจะเป็นของอารมณ์ต่าง ๆ 35
รูปผนวกที่ ก.1 สีของอารมณ์ต่าง ๆ 38
รูปผนวกที่ ข.1 หน้าเว็บไซต์เพื่อดาวน์โหลดไฟล์ 39
รูปผนวกที่ ข.2 หน้า License Agreement 39
รูปผนวกที่ ข.3 หน้าเลือกที่อยู่ไฟล์ 40
รูปผนวกที่ ข.4 หน้าเลือกโฟลเดอร์เก็บทางลัดโปรแกรมที่จะติดตั้ง 40
รูปผนวกที่ ข.5 หน้าเลือกการตั้งค่าเพิ่มเติมที่ต้องการในระหว่างการติดตั้งโปรแกรม 40
รูปผนวกที่ ข.6 หน้าแสดงรายละเอียดเกี่ยวกับการติดตั้งโปรแกรม 41
รูปผนวกที่ ข.7 รอดำเนินการจนเสร็จสิ้น 41
รูปผนวกที่ ข.8 หน้าแสดงสถานะการติดตั้ง 42
รูปผนวกที่ ข.9 หน้า interface ของ Visual Studio Code 42
รูปผนวกที่ ข.10 ไลบรารี่ที่ติดตั้ง 43

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

การทำให้ผู้ใช้รู้สึกและเข้าใจอารมณ์ที่แท้จริงของตนเองต่อคลิปวิดีโอที่ชม ได้มีความสำคัญอย่างมากในการวิจัยและพัฒนาสื่อและเทคโนโลยีที่เป็นมิตรต่อผู้ใช้ โดยปกติแล้วผู้ชมอาจจะมีการรับรู้และประเมินความรู้สึกต่อคลิปวิดีโอในระหว่างการรับชม แต่ความรู้สึกนี้อาจจะไม่ได้บ่งชี้อย่างชัดเจน

ทางผู้วิจัยได้เห็นถึงความสำคัญ จึงเน้นการวิเคราะห์อารมณ์จากคลิปวิดีโอ เพื่อทำให้ผู้ใช้รับรู้และเข้าใจอารมณ์ที่แท้จริงของตนเองต่อคลิปวิดีโอที่ชมได้ เป็นแนวทางที่สำคัญในการพัฒนาสื่อและเทคโนโลยีในอนาคต เพื่อสร้างประสบการณ์การรับชมที่มีคุณภาพและเป็นประโยชน์ต่อผู้ใช้โดยตรง

1.2 วัตถุประสงค์

1) เพื่อทำความเข้าใจว่าผู้ชมตอบสนองทางอารมณ์ต่อเนื้อหาประเภทต่าง ๆ อย่างไร
2) เพื่อทดสอบตอนหรือฉากนำร่อง เพื่อวัดปฏิกิริยาของผู้ชมและปรับเนื้อหาให้เหมาะสม

1.3 ลักษณะและขอบเขต

โครงการนี้ใช้การตรวจจับใบหน้าใช้เทคโนโลยี machine learning (ML) และเทคโนโลยีโครงข่ายประสาทเทียม (ANN) มีบทบาทสำคัญในการติดตามใบหน้า การวิเคราะห์ใบหน้า และการจดจำใบหน้า ในการวิเคราะห์ใบหน้า การตรวจจับใบหน้าใช้การแสดงออกทางสีหน้าเพื่อระบุว่า ผู้ใช้กำลังมีความรู้สึกใด

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

- เครื่องคอมพิวเตอร์

1.4.2 ซอฟต์แวร์

- Visual Studio Code
- Python
- Google Colab

1.5 ผลที่คาดว่าจะได้รับ

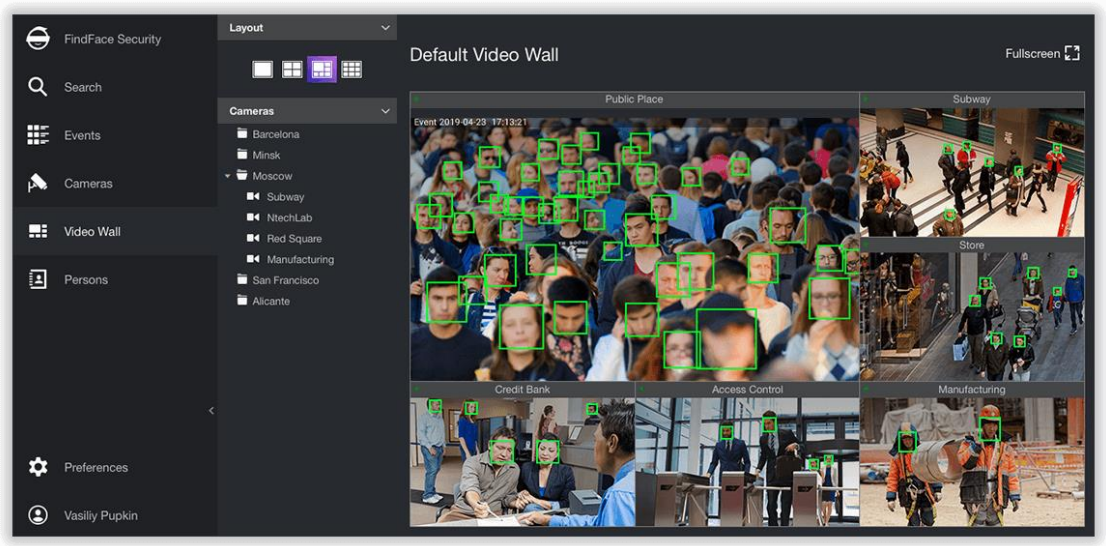
1) ระบบสามารถดูได้ว่าช่วงไหนรู้สึกอย่างไรบ้าง
2) ระบบสามารถสรุปได้ว่าโดยรวมผู้ใช้รู้สึกอย่างไรบ้าง โดยประมวลผลลัพธ์ออกเป็น (ชนิดอารมณ์ต่าง ๆ, ตัวเลขบ่งชี้ชัดเจน และกราฟ)
3) สามารถวิเคราะห์ได้ทั้งวิดีโอ และเรียลไทม์
4) ระบบสามารถจัดเก็บความรู้สึกในช่วงเวลาต่าง ๆ ลงในฐานข้อมูล
5) ระบบสามารถใช้งานและเข้าใจได้ง่าย เหมาะกับผู้ใช้เริ่มต้น

บทที่ 2 ผลงานที่เกี่ยวข้อง

ในส่วนของผลงานที่เกี่ยวข้องประกอบด้วย 5 ผลงาน ที่นำมาใช้เป็นแนวทางการพัฒนาโปรแกรมเพื่อช่วยตรวจจับและวิเคราะห์อารมณ์จากใบหน้า ซึ่งแต่ละผลงานที่นำมามีความคล้ายคลึงกับโปรแกรมนี้ โดยจะอธิบายถึงคุณสมบัติ ข้อดี และข้อเสีย

2.1 Findface

ใช้อัลกอริทึมการจดจำใบหน้าของ NtechLab เพื่อจดจำอารมณ์พื้นฐาน 7 อารมณ์และคุณลักษณะที่ซับซ้อน 50 รายการ มีความแม่นยำถึง 94% โดยสามารถรับรู้อารมณ์ 7 อารมณ์ ได้แก่ ความสุข ความประหลาดใจ ความเศร้า ความโกรธ ความรังเกียจ การดูถูก และความกลัว



รูปที่ 2.1 Findface

ข้อดี / ฟีเจอร์ / คุณสมบัติ	ข้อเสีย
✓ มักใช้ในบริบทการบังคับใช้กฎหมายและความปลอดภัย สามารถช่วยระบุตัวบุคคลในพื้นที่แออัดหรือติดตามผู้ต้องสงสัยได้ ✓ การระบุตัวตนแบบเรียลไทม์ที่รวดเร็วและมีประสิทธิภาพและความทนทานต่อข้อผิดพลาดในระบบโหลดสูงด้วยกล้อง	✗ ใช้งานค่อนข้างยาก สำหรับผู้ใช้ใหม่ ✗ ต้องมีความชำนาญถึงจะใช้ได้ ✗ ไม่สามารถระบุได้ชัดเจนว่า ช่วงเวลาไหนผู้ใช้กำลังมีความรู้สึกใด

2.2 EmoVu

เป็นระบบหรือแพลตฟอร์มที่ช่วยให้สามารถวัดการมีส่วนร่วมทางอารมณ์ของเนื้อหาวิดีโอและประสิทธิผลต่อผู้ชมเป้าหมายได้อย่างแม่นยำ ซอฟต์แวร์บนคลาวด์จะอ่านการแสดงสีหน้า ผ่านเว็บแคม ในขณะที่ผู้ชมที่เป็นเป้าหมายรับชมเนื้อหาวิดีโอบนคอมพิวเตอร์หรืออุปกรณ์พกพา จากนั้นข้อมูลทางอารมณ์จะถูกซิงโครไนซ์โดยอัตโนมัติกับการเล่นวิดีโอสำหรับวิเคราะห์แบบวินาทีต่อวินาที

รูปที่ 2.2 Emovu

ข้อดี / ฟีเจอร์ / คุณสมบัติ	ข้อเสีย
✓ สร้าง EmoVu REST API เป็น API การจดจำอารมณ์แบบการเรียนรู้เชิงลึกที่สามารถใช้เพื่อตรวจจับการแสดงออกทางสีหน้าในรูปภาพและทำการวิเคราะห์ความรู้สึก ✓ สามารถตรวจจับอารมณ์ได้ทั้งภาพ วิดีโอและเรียลไทม์ ✓ ดูผลการตรวจจับได้แบบวินาทีต่อวินาที	✘ ชุดฟีเจอร์ที่จำกัด อาจไม่ยืดหยุ่นพอที่จะสร้างแอพพลิเคชั่นที่ซับซ้อน ✘ ใช้งานยากสำหรับผู้ใช้เริ่มต้น ✘ ไม่สามารถระบุได้ชัดเจนว่า ช่วงเวลาไหนผู้ใช้กำลังมีความรู้สึกใด

2.3 iMotions

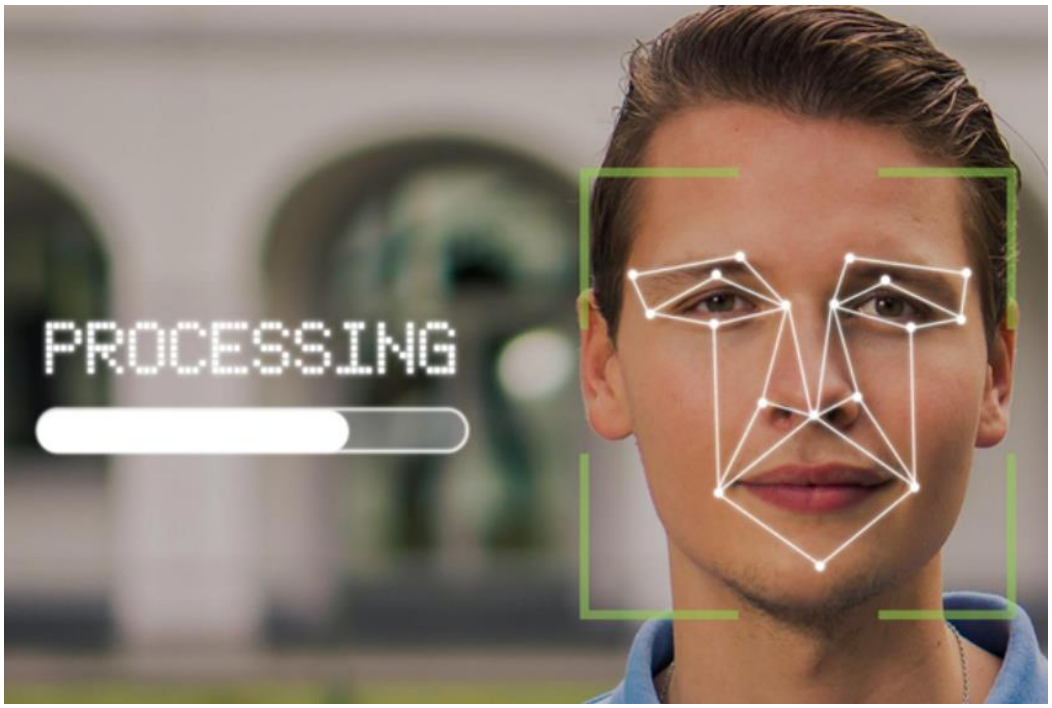
เป็นซอฟต์แวร์ที่ผสานรวมและซิงโครไนซ์สัญญาณทางสรีรวิทยาจากไบโอเซนเซอร์หลายตัว โดยแต่ละตัวให้ข้อมูลเชิงลึกที่แตกต่างกันเกี่ยวกับสรีรวิทยาและอารมณ์ของมนุษย์ ซึ่งรวมถึงการติดตามดวงตา, EDA/GSR, EEG, ECG และการวิเคราะห์การแสดงออกทางสีหน้า iMotions รวบรวมเทคโนโลยีฮาร์ดแวร์ที่จำเป็นทั้งหมดและข้อมูลที่เกี่ยวข้องมาไว้ในเส้นทางเดียวที่สอดคล้องกันเพื่อให้ทำงานร่วมกันได้อย่างราบรื่น

รูปที่ 2.3 iMotions

ข้อดี / ฟีเจอร์ / คุณสมบัติ	ข้อเสีย
✓ สามารถประมวลผลและวิเคราะห์ข้อมูลแบบเรียลไทม์ ✓ สามารถรวบรวมและซิงโครไนซ์ข้อมูลจากเซ็นเซอร์และอุปกรณ์ไบโอเมตริกซ์ต่าง ๆ ได้พร้อมกัน ✓ สามารถจัดการกับการทดลองที่มีขนาดและความซับซ้อนที่แตกต่างกัน ทำให้เหมาะสำหรับโครงการวิจัยที่หลากหลาย ตั้งแต่การศึกษาขนาดเล็กไปจนถึงการทดลองขนาดใหญ่	✗ กราฟไม่สามารถวิเคราะห์ระดับความรู้สึกได้ชัดเจนว่าอยู่ที่ระดับใด และผู้ใช้อาจจะอ่านผลลัพธ์ไม่เข้าใจ ✗ iMotions และอุปกรณ์ที่ใช้ร่วมกัน (เช่น กล้องสำหรับติดตามดวงตา เซ็นเซอร์ EEG) มีราคาค่อนข้างสูง

2.4 SkyBiometry

เป็นเครื่องมือตรวจจับและจดจำใบหน้าบนคลาวด์ ซึ่งช่วยให้คุณตรวจจับอารมณ์ในภาพถ่ายได้ อัปโหลดไฟล์ แล้วจะทำการตรวจจับใบหน้า และสัมผัสอารมณ์ระหว่างความสุข เศร้า โกรธ ประหลาดใจ รังเกียจ กลัว และเป็นกลาง โดยมีอัตราเปอร์เซ็นต์สำหรับแต่ละจุด กำหนดได้อย่างแม่นยำว่าบุคคลนั้นกำลังยิ้มหรือไม่

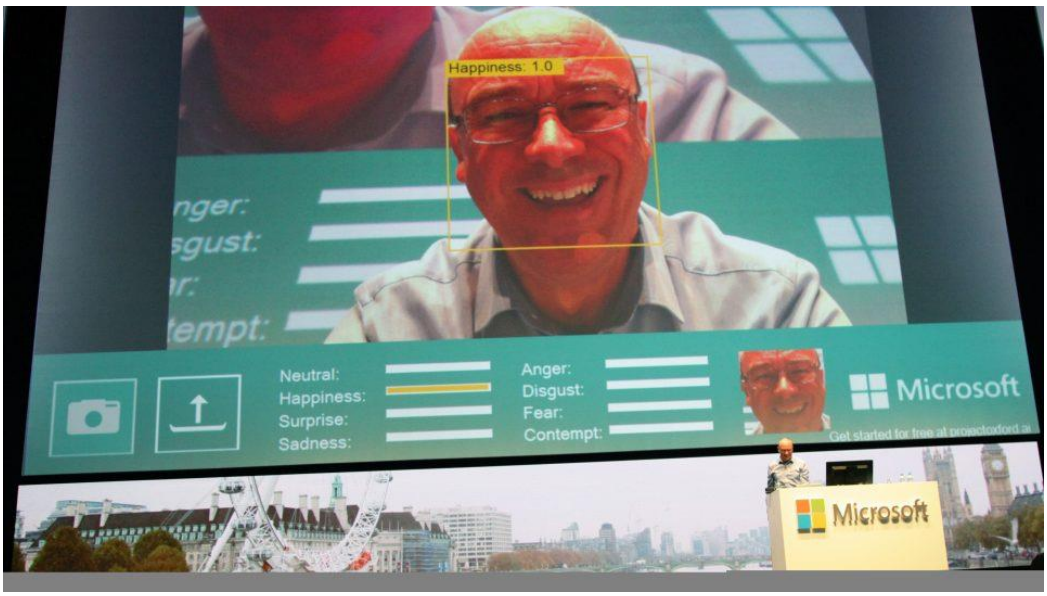


รูปที่ 2.4 SkyBiometry

ข้อดี / ฟีเจอร์ / คุณสมบัติ	ข้อเสีย
✓ API ของ SkyBiometry สามารถใช้กับภาษาการเขียนโปรแกรมและแพลตฟอร์มต่าง ๆ ได้ ซึ่งให้ความยืดหยุ่นในการพัฒนาและการใช้งาน ✓ สามารถตรวจจับและจดจำใบหน้าหลายใบหน้าภายในภาพหรือเฟรมวิดีโอเดียว ✓ ทำงานเป็นบริการบนคลาวด์ ช่วยให้ขยายขนาดได้ง่าย ✓ สามารถรองรับคำขอปริมาณมากได้ ทำให้เหมาะสำหรับการใช้งานที่มีความต้องการปริมาณงานสูง	✗ อาจไม่เหมาะกับทุกการใช้งาน เช่น อาจไม่มีประสิทธิภาพสำหรับการใช้งานทางไกลหรือแบบเรียลไทม์เมื่อเปรียบเทียบกับเทคโนโลยีไบโอเมตริกซ์อื่น ๆ ✗ ตรวจจับใบหน้าและจำแนกประเภทอารมณ์ได้แค่ภาพถ่าย จึงอาจจะไม่เหมาะกับการวิเคราะห์ผ่านคลิปวิดีโอ หรือแบบเรียลไทม์ต่าง ๆ

2.5 Microsoft Project Oxford

เป็นเครื่องมือของ Microsoft สามารถใช้เพื่อสร้างระบบที่รับรู้สภาวะทางอารมณ์หลักแปดสภาวะ ได้แก่ ความโกรธ ดูถูก ความกลัว ความรังเกียจ ความสุข เป็นกลาง ความเศร้า หรือความประหลาดใจ โดยยึดตามการแสดงออกทางสีหน้าสากล



รูปที่ 2.5 SkyBiometry

ข้อดี / ฟีเจอร์ / คุณสมบัติ	ข้อเสีย
✓ มี Face API ที่สามารถตรวจจับและจดจำใบหน้ามนุษย์จากรูปภาพได้ ✓ มีความสามารถในการประมวลผลข้อความและคำพูดในหลายภาษา ทำให้เหมาะสำหรับแอปพลิเคชันที่มีฐานผู้ใช้ทั่วโลก ✓ สามารถตรวจจับอารมณ์ได้ทั้งภาพ วิดีโอ และเรียลไทม์	✘ โมเดล AI รวมถึงโมเดลจาก Microsoft อาจไม่ได้ให้ผลลัพธ์ที่แม่นยำสมบูรณ์แบบเสมอไป ความแม่นยำอาจแตกต่างกันไปขึ้นอยู่กับงานเฉพาะ คุณภาพของข้อมูล และรุ่นที่ใช้ ✘ ไม่ได้บอกความชัดเจนในรายละเอียดของอารมณ์

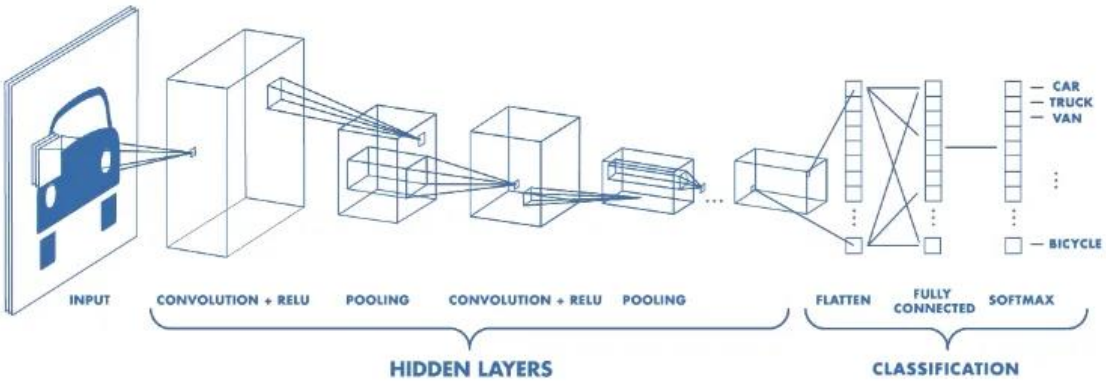
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

การจัดทำโครงงานฉบับนี้ผู้พัฒนาได้ค้นคว้าข้อมูลทฤษฎีและความรู้ที่เกี่ยวข้องโดยอาศัยความสำคัญต่อการออกแบบและพัฒนาระบบประกอบไปด้วย

3.1 Convolutional Neural Networks (CNN)

Convolutional Neural Networks เป็นแบบจำลองการเรียนรู้ของเครือข่ายประสาทเทียม (Neural Network) ที่ถูกออกแบบมาเพื่อการประมวลผลภาพและวิดีโออย่างเฉพาะเจาะจง โดยมีคุณสมบัติหลัก ๆ คือการใช้ชั้นการสังเคราะห์ (Convolutional Layers) เพื่อสกัดลักษณะหรือลายเส้นพิเศษจากข้อมูลภาพ ซึ่งชั้นนี้ช่วยให้ CNN เหมาะสมและมีประสิทธิภาพในการจัดการกับข้อมูลภาพเป็นพิเศษ โดยทั่วไปจะใช้ในงาน computer vision เช่น การจดจำวัตถุ การจำแนกภาพ และการตรวจจับใบหน้า ได้รับการฝึกฝนให้จดจำและตีความลักษณะใบหน้าและการแสดงออกเพื่อกำหนดอารมณ์ได้อย่างแม่นยำ และจัดการกับข้อมูลที่ซับซ้อนได้ดี

CNN เป็นเครื่องมือที่มีประสิทธิภาพสำหรับงานที่เกี่ยวข้องกับภาพ เช่น การจำแนกและตรวจจับวัตถุในภาพ การระบุอารมณ์ใบหน้า การจดจำตัวเลขหรือตัวหนังสือ และอื่น ๆ ที่เกี่ยวข้องกับการประมวลผลภาพและวิดีโอ [1]

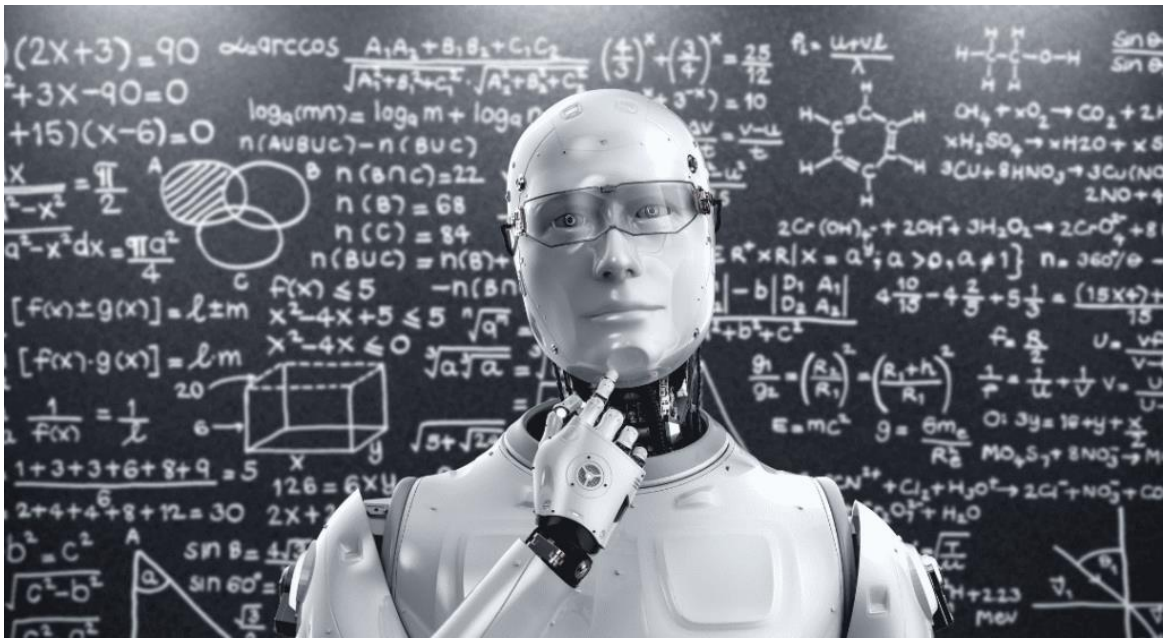


รูปที่ 3.1 Convolutional Neural Networks

3.2 machine learning (ML)

Machine Learning คือ กระบวนการ การเรียนรู้ของโปรแกรมคอมพิวเตอร์ มีหลักการคล้าย ๆ กับของมนุษย์ที่จำเป็นต้องเรียนรู้จากประสบการณ์ ด้วยการป้อนชุดข้อมูลพื้นฐานและชุดคำสั่งต่าง ๆ เพื่อให้คอมพิวเตอร์ “เรียนรู้” และจำแนกแยกแยะวัตถุต่าง ๆ รวมถึงบุคคล สิ่งของ เป็นต้น และเพื่อให้ได้ผลลัพธ์ที่แม่นยำมากขึ้น โปรแกรมเมอร์จึงมีการป้อนชุดข้อมูลใหม่ ๆ และต้องฝึกฝนระบบอย่างสม่ำเสมอ เพื่อให้เกิดวิวัฒนาการของระบบ ทำให้ระบบคิดได้ด้วยตัวเองในเวลาต่อมา โดยที่มนุษย์ไม่ต้องเขียนโปรแกรมเพิ่มหรือกำกับการใช้งาน

ระบบ Machine Learning จะใช้เทคนิคทางสถิติเพื่อหาความสัมพันธ์ระหว่างข้อมูล วิเคราะห์การกระจายของข้อมูล และคำนวณค่าต่าง ๆ เช่น ค่าเฉลี่ย ค่าความแปรปรวน หรือค่าความสัมพันธ์ เพื่อให้เข้าใจลึกซึ้งเกี่ยวกับข้อมูลและภาพรวมของปัญหาที่ต้องการแก้ไข โดยมีวัตถุประสงค์หลักคือการปรับปรุงการทำนายและการสร้างโมเดลที่มีประสิทธิภาพมากที่สุด การปรับปรุงนี้มักจะเกิดขึ้นร่วมกับการทดสอบและปรับแก้โมเดลอย่างต่อเนื่อง เพื่อให้สามารถสร้างโมเดลที่มีประสิทธิภาพและแม่นยำในการทำนายผลลัพธ์ได้มากที่สุด [2]



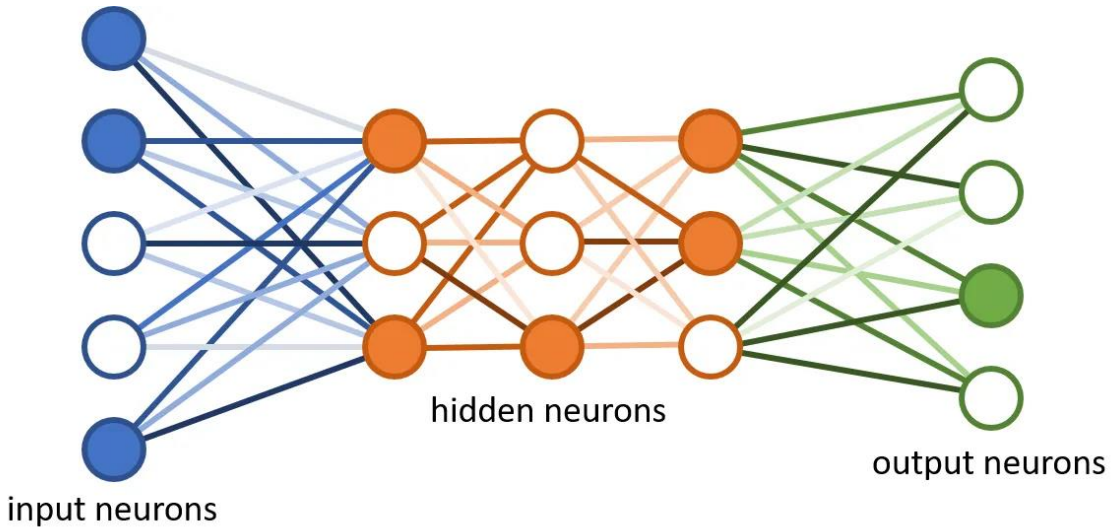
รูปที่ 3.2 machine learning

3.3 Artificial Neural Network (ANN)

หรือ "เครือข่ายประสาทเทียม" เป็นเครื่องมือที่สำคัญในด้านปัญญาประดิษฐ์และการเรียนรู้เชิงลึก เนื่องจากมันมีความสามารถในการจำลองโครงสร้างของระบบประสาทในสมองมนุษย์ ทำให้สามารถนำมาใช้ในการแก้ไขปัญหาที่ซับซ้อนและหลากหลายในหลายสาขา เช่น การประมวลผลภาพ การระบุวัตถุ การจำแนกประเภทของข้อมูล การทำนายผลลัพธ์ และการวิเคราะห์ข้อมูลที่ซับซ้อน

ANN มีลักษณะการทำงานที่คล้ายกับการทำงานของเซลล์ประสาทในสมอง โดยประกอบด้วยชั้นของโหนดหรือเซลล์ประสาทที่เชื่อมต่อกันและส่งข้อมูลระหว่างกัน โดยโหนดแต่ละตัวจะรับข้อมูลจากโหนดก่อนหน้าและประมวลผลเพื่อส่งต่อไปยังโหนดถัดไป ซึ่งกระบวนการนี้จะซ้ำไปเรื่อย ๆ จนถึงชั้นสุดท้ายที่ได้ผลลัพธ์ที่เป็นคำตอบของปัญหา

นอกจากนี้ ANN ยังมีความยืดหยุ่นและหลากหลายมากเพราะสามารถปรับปรุงโครงสร้างและพารามิเตอร์ต่าง ๆ ของโมเดลได้ตามความต้องการของงาน เช่น การเพิ่มหรือลดจำนวนชั้นของโหนด การเปลี่ยนแปลงตัวแปรอินพุต หรือการเปลี่ยนแปลงฟังก์ชันการเรียนรู้ ดังนั้น ANN เป็นเครื่องมือที่สำคัญในการพัฒนาและการแก้ไขปัญหาที่ซับซ้อนและหลากหลายในโลกปัจจุบัน และมีการประยุกต์ใช้ในหลายสาขาที่มีความสำคัญในการพัฒนาสังคมและเศรษฐกิจ [3]

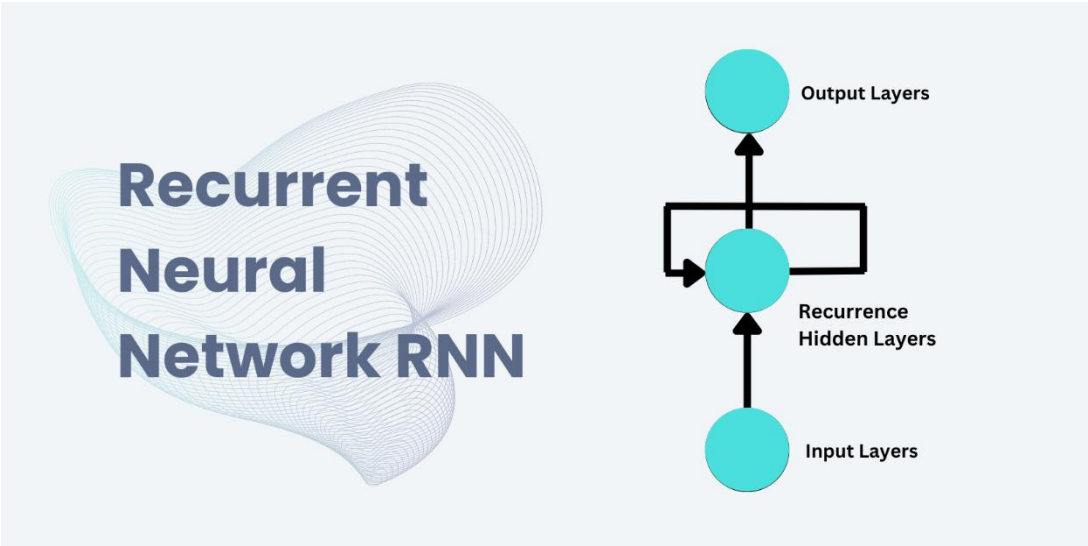


รูปที่ 3.3 Artificial Neural Network

3.4 Recurrent Neural Network (RNN)

หรือ "เครือข่ายประสาทที่ทำงานแบบซ้ำซ้อน" ซึ่งเป็นโมเดลทางคอมพิวเตอร์ที่ถูกออกแบบมาเพื่อการประมวลผลข้อมูลที่มีลำดับหรือเป็นชุดเวลา (time series data) โดยมีความสามารถในการจำความสัมพันธ์ข้ามกันระหว่างข้อมูลในลำดับต่าง ๆ ทำให้เหมาะสำหรับการทำนาย การแปลความหมายของประโยค หรืองานที่เกี่ยวกับข้อมูลที่มีลำดับ

RNN เป็นเครื่องมือที่มีความสำคัญในการประมวลผลข้อมูลที่มีลำดับและเป็นเครื่องมือที่มีความยืดหยุ่นในการปรับปรุงโมเดลให้เหมาะสมกับงานที่ต้องการ [4]



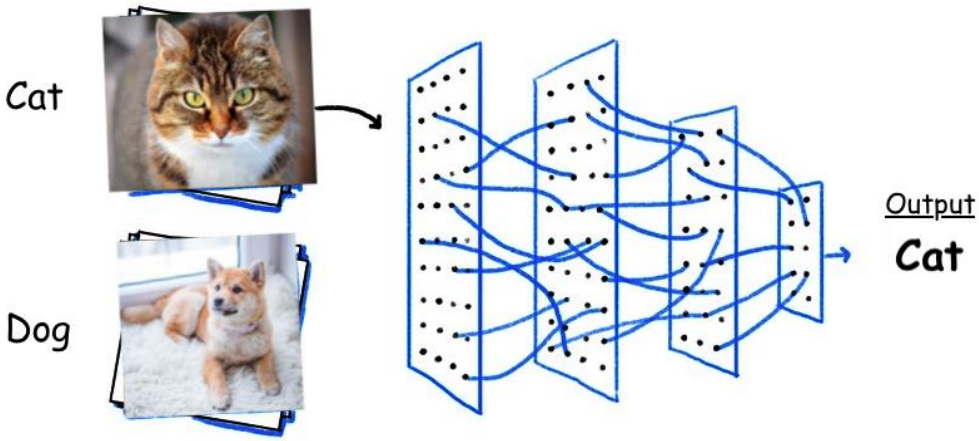
รูปที่ 3.4 Recurrent Neural Network

3.5 Image Classification

เป็นกระบวนการที่สำคัญในการวิเคราะห์ภาพดิจิทัล เพื่อระบุหรือจำแนกประเภทของวัตถุหรือสิ่งของในภาพ โดยการใช้เทคโนโลยีทางคอมพิวเตอร์ เช่น การใช้โมเดลเรียนรู้เชิงลึก (deep learning) หรือการประมวลผลภาพ (image processing) ซึ่งมีความสำคัญในหลายด้าน:

- การจำแนกประเภทของวัตถุ: Image Classification ช่วยในการแยกแยะวัตถุในภาพออกเป็นกลุ่มหรือประเภทต่าง ๆ ที่ต้องการ เช่น รถยนต์, คน, สัตว์, วัตถุทั่วไป เป็นต้น ซึ่งเป็นส่วนสำคัญในการสร้างระบบการจดจำและการตรวจจับวัตถุอัตโนมัติ
- การสร้างระบบการประมวลผลภาพอัตโนมัติ: Image Classification เป็นส่วนสำคัญในการสร้างระบบตรวจจับวัตถุหรือแยกแยะวัตถุต่าง ๆ ในภาพอัตโนมัติ เช่น ระบบตรวจจับใบหน้าในภาพ หรือการจัดเรียงภาพตามหมวดหมู่ที่แตกต่างกัน
- การวิเคราะห์และเข้าใจข้อมูล: Image Classification ช่วยให้สามารถวิเคราะห์และเข้าใจข้อมูลที่มาจากภาพได้มากขึ้น เช่น การวิเคราะห์ข้อมูลทางการแพทย์ เพื่อจำแนกภาพเป็นภาพที่แสดงถึงโรคหรือความผิดปกติต่าง ๆ นอกจากนี้ ยังช่วยในการวิเคราะห์ภาพที่ใช้ในการเข้าใจปัญหาและการตัดสินใจในหลาย ๆ สถานการณ์

Image Classification เป็นเครื่องมือที่สำคัญและมีประโยชน์ในการประมวลผลภาพเพื่อเพิ่มความเข้าใจและประสิทธิภาพในการวิเคราะห์ข้อมูลและตัดสินใจในหลาย ๆ สถานการณ์ชีวิตประจำวันและในงานอาชีพที่ต้องการการวิเคราะห์ภาพอย่างเป็นระบบ [5]

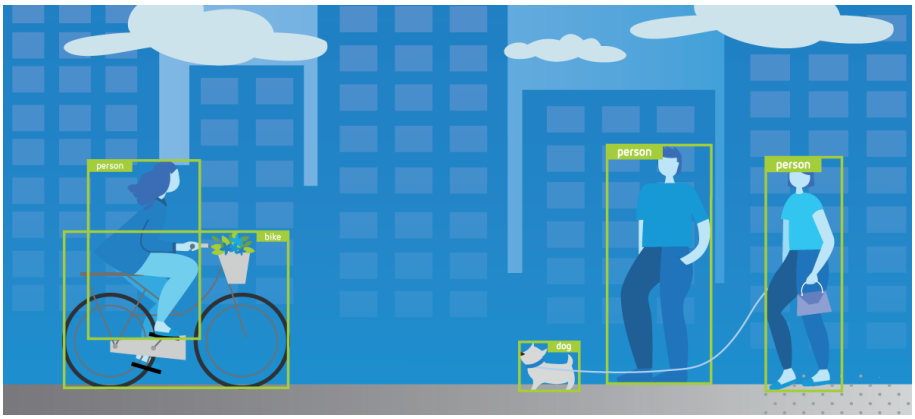


รูปที่ 3.5 Image Classification

3.6 Image Detection

Image detection หมายถึงกระบวนการในการตรวจจับวัตถุ หรือลักษณะต่าง ๆ ในภาพดิจิทัลด้วยการใช้เทคโนโลยีของคอมพิวเตอร์ เช่นการใช้โมเดลเรียนรู้เชิงลึก (deep learning) หรือการประมวลผลภาพ (image processing) เพื่อระบุวัตถุหรือลักษณะที่สนใจในภาพ ตัวอย่างของงานที่ใช้ Image detection ได้แก่การระบุวัตถุในภาพถ่าย เช่น รถยนต์, คน, สัตว์, และวัตถุทั่วไป การตรวจจับใบหน้าในภาพ, การแยกแยะและจำแนกประเภทของวัตถุในภาพ, การตรวจจับสิ่งของที่เป็นอันตรายในภาพเพื่อความปลอดภัย เป็นต้น

การใช้ Image detection มีความสำคัญในหลายแวดวง เช่น ในงานด้านการวิเคราะห์ภาพทางการแพทย์ เพื่อตรวจจับโรคหรือการบาดเจ็บ, ในงานด้านการยานยนต์เพื่อระบุรถเยอรมัน, ในงานด้านความปลอดภัยเพื่อตรวจจับพฤติกรรมที่ผิดปกติ, และในงานด้านการตรวจสอบความถูกต้องของผลิตภัณฑ์ และการทำขนาดใหญ่โดยใช้เทคโนโลยีอัจฉริยะในการตรวจสอบความถูกต้องของสินค้าที่ผลิต [6]



รูปที่ 3.6 Image Detection

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

ผู้พัฒนาได้วางแผน และเริ่มดำเนินงานเป็นขั้นตอนเพื่อจัดทำโครงงานฉบับนี้ขึ้น โดยผู้พัฒนาจะแสดงขั้นตอนการดำเนินงาน และอธิบายรายละเอียดขั้นตอนการดำเนินงาน

4.1 ขั้นตอนการดำเนินงาน

4.1.1 การคิดหัวข้อโครงงานและรวบรวมข้อมูล

- เลือกหัวข้อที่สนใจ และรวบรวมข้อมูลที่เกี่ยวข้อง
- นำเสนอแนวคิด และหัวข้อที่ผู้พัฒนาสนใจ กับอาจารย์ที่ปรึกษาโครงงาน

4.1.2 กำหนดขอบเขตและวางแผนการดำเนินงาน

- กำหนดขอบเขตของโครงงาน
- วางแผนการดำเนินงานและระยะเวลาในการดำเนินงาน

4.1.3 ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง

- ศึกษาทฤษฎีที่ต้องใช้เกี่ยวกับโครงงาน
- ศึกษาการใช้ Python
- ศึกษาการใช้ Colab

4.1.4 ออกแบบระบบงาน

- กำหนดขอบเขตของโครงงาน
- วางแผนการดำเนินงานและระยะเวลาในการดำเนินงาน

4.1.5 การพัฒนาระบบ

- ติดตั้งโปรแกรมที่นำมาใช้ในการพัฒนาโครงงาน
- ทดสอบโปรแกรมที่ใช้งาน
- สร้างฐานข้อมูลเพื่อดึงข้อมูลผลลัพธ์มาเก็บไว้

4.1.6 การทดสอบระบบ

- ตรวจสอบความถูกต้อง และการทำงานของระบบ
- ปรับปรุง และแก้ไข

4.1.7 จัดทำเอกสารประกอบโครงงาน

- รวบรวม และเรียบเรียงข้อมูลที่เกี่ยวกับโครงงานทั้งหมด ตั้งแต่เริ่มต้นขั้นตอนการจัดทำโครงงาน จนถึงขั้นตอนสุดท้ายของการจัดทำโครงงาน

4.2 แผนการดำเนินงาน

ตารางที่ 1 แผนการดำเนินงาน

ขั้นตอนการดำเนินงาน	เดือน							
	เม.ย. 2567	พ.ค. 2567	มิ.ย. 2567	ก.ค. 2567	ส.ค. 2567	ก.ย. 2567	ต.ค. 2567	พ.ย. 2567
การคิดหัวข้อโครงงานและรวบรวมข้อมูล	↔							
กำหนดขอบเขตและวางแผนการดำเนินงาน	↔	↔						
ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง	↔	↔	↔					
ออกแบบระบบงาน				↔	↔	↔	↔	
การพัฒนาระบบ						↔	↔	
การทดสอบระบบ							↔	↔
จัดทำเอกสารประกอบโครงงาน		↔	↔	↔	↔	↔	↔	↔

บทที่ 5 การวิเคราะห์และออกแบบระบบ

ในบทนี้จะกล่าวถึงกระบวนการดำเนินงานของระบบ โครงสร้างการทำงานของระบบว่ามีลักษณะการทำงานอย่างไร โดยแบ่งเป็นลักษณะดังนี้

5.1 ภาพรวมของที่มาของโค้ดและเทคนิคการนำมาใช้ในระบบ

5.1.1 ที่มาของโค้ดและเทคนิคการนำมาใช้ในระบบ

การพัฒนาระบบสำหรับการวิเคราะห์และติดตามอารมณ์ของผู้ชมจากเนื้อหาวิดีโอในงานวิจัยนี้ อ้างอิงแนวคิดจากบทความ "Frame-by-Frame Tracking Emotions in Videos with AI" โดย John Solomon Legara ซึ่งเป็นส่วนสำคัญในการสร้างระบบที่สามารถตรวจจับและระบุอารมณ์ของผู้ชมได้อย่างแม่นยำในแต่ละเฟรมของวิดีโอ โดยโค้ดและขั้นตอนที่อ้างอิงมาจากบทความนี้ประกอบด้วยรายละเอียดดังนี้

5.1.1.1 การใช้ MTCNN สำหรับการตรวจจับอารมณ์

บทความแนะนำการใช้โมเดล MTCNN (Multi-task Cascaded Convolutional Networks) เป็นอัลกอริทึมการตรวจจับใบหน้าที่ใช้โครงข่ายประสาทเทียมแบบคอนโวลูชัน (Convolutional Neural Networks - CNN) หลายขั้นตอนเพื่อตรวจจับใบหน้าในภาพอย่างมีประสิทธิภาพ โดย MTCNN ถูกออกแบบมาเพื่อทำงานหลายอย่าง (multi-task) ในเวลาเดียวกัน เช่น การตรวจจับขอบเขตของใบหน้าและการระบุจุดสำคัญ (landmarks) บนใบหน้า เช่น ตา จมูก และปาก

```
# เริ่มต้นแบบจำลอง MTCNN สำหรับการครอบตัดหน้าเดียว
 mtcnn = MTCNN(
    image_size= 160 ,
    margin= 0 ,
    min_face_size= 200 ,
    thresholds=[ 0.6 , 0.7 , 0.7 ],
    factor= 0.709 ,
    post_process= True ,
    keep_all= False ,
    device=device
)
```

รูปที่ 5.1 การใช้ Machine Learning Model

5.1.1.2 โมเดลที่นำมาใช้งาน

```
# โหลดแบบจำลองที่ได้รับการฝึกล่วงหน้าและตัวแยกคุณลักษณะ extractor
 = AutoFeatureExtractor.from_pretrained(
    "trpakov/vit-face-expression"
 )
model = AutoModelForImageClassification.from_pretrained(
    "trpakov/vit-face-expression"
 )
```

รูปที่ 5.2 โมเดลที่นำมาใช้งาน

5.1.1.3 การแบ่งเฟรมและการประมวลผลแบบเฟรมต่อเฟรม

วิธีการประมวลผลวิดีโอแบบเฟรมต่อเฟรมถูกนำมาใช้ในการแยกแต่ละเฟรมออกมาเพื่อตรวจสอบอารมณ์ของผู้ชมในช่วงเวลาต่าง ๆ ของวิดีโอ ซึ่งการทำงานแบบเฟรมต่อเฟรมนี้ช่วยให้สามารถวิเคราะห์การเปลี่ยนแปลงของอารมณ์ได้อย่างละเอียด ฟังก์ชันการตรวจจับอารมณ์ในเฟรมต่าง ๆ ของวิดีโอ โดยจะเลือกแบบสุ่มเพื่อให้ได้ตัวอย่างวิดีโอที่หลากหลาย สำหรับแต่ละเฟรมที่เลือก ให้ทำตามขั้นตอนต่อไปนี้

- เฟรมจะถูกดึงมาจากข้อมูลวิดีโอ (ซึ่งจะถูกเก็บไว้เป็นรายการของอาร์เรย์ NumPy ที่แสดงถึงเฟรมแต่ละเฟรม)
- เฟรมจะถูกแปลงเป็นPILวัตถุ Image ซึ่งจำเป็นเนื่องจากฟังก์ชัน detect_emotions คาดหวังอินพุตในรูปแบบ PILImage
- ฟังก์ชัน detect_emotionsถูกเรียกใช้งานโดยมี PILImage เป็นอินพุต ฟังก์ชันนี้จะตรวจจับใบหน้าในภาพ ทำนายความน่าจะเป็นของอารมณ์ต่าง ๆ

5.1.1.4 การนำ OpenCV มาใช้ในการประมวลผลภาพและตรวจจับใบหน้า

บทความนี้ยังอธิบายถึงการใช้ OpenCV ไลบรารีที่มีเครื่องมือสำหรับการประมวลผลภาพ ซึ่งสามารถนำมาปรับใช้ในการตรวจจับใบหน้าในวิดีโอ ทำให้ระบบสามารถระบุและติดตามอารมณ์ที่แสดงออกบนใบหน้าได้อย่างชัดเจน

5.1.1.5 การลดจำนวนเฟรมเพื่อลดระยะเวลาประมวลผลในการวิเคราะห์อารมณ์จากวิดีโอ

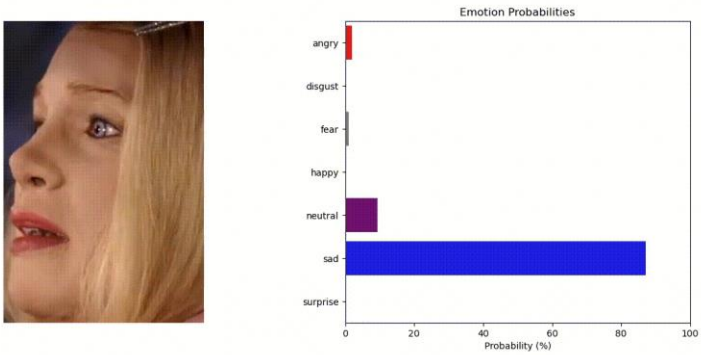
กระบวนการนี้เรียกว่าการข้ามเฟรม (Frame skip) ซึ่งจะช่วยเพิ่มประสิทธิภาพการคำนวณของการดำเนินการในภายหลัง เช่น การตรวจจับอารมณ์ ซึ่งอาจต้องใช้ทรัพยากรมาก skips ตัวแปรจะกำหนดจำนวนเฟรมที่จะข้ามหลังจากเลือกแต่ละเฟรม

```
skips = 2
 dropped_video = []

สำหรับ i ใน tqdm( ช่วง ( 0 , len (video_data), skips)):
    dropped_video.append(video_data[i])
```

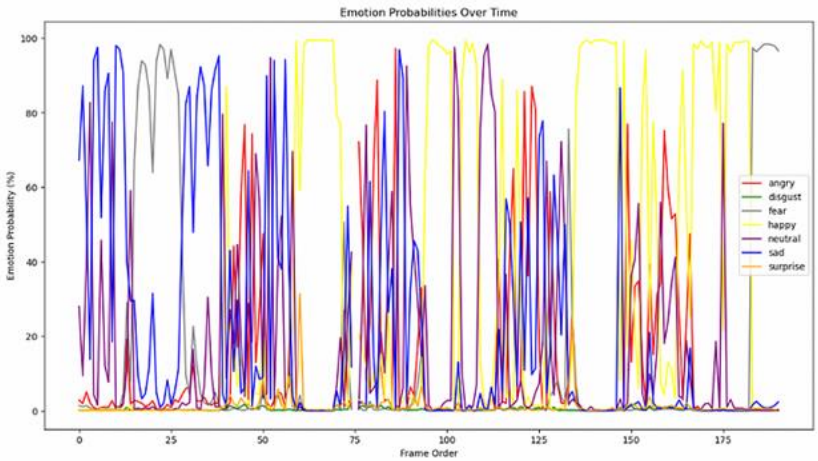
รูปที่ 5.3 การข้ามเฟรม (Frame skip)

5.1.1.6 การนำไปประยุกต์ใช้โค้ดและขั้นตอนในระบบการทำงาน

โค้ดและเทคนิคที่อ้างอิงจากบทความนี้ได้ถูกนำมาปรับใช้ในระบบเพื่อทำให้การตรวจจับอารมณ์และการประมวลผลมีความแม่นยำมากขึ้น โดยการใช้งานเฟรมต่อเฟรมช่วยให้การวิเคราะห์อารมณ์เกิดขึ้นได้อย่างต่อเนื่อง อีกทั้งยังสามารถระบุช่วงเวลาและลำดับของการแสดงอารมณ์ในวิดีโอได้อย่างมีประสิทธิภาพ



รูปที่ 5.4 การนำไปประยุกต์ใช้โค้ดและขั้นตอนในระบบการทำงาน



รูปที่ 5.5 ช่วงเวลาและลำดับการแสดงอารมณ์

5.2 ภาพรวมการทำงานและการทำงานของระบบ

5.2.1 โครงสร้างของระบบ

ระบบมีการแบ่งออกเป็นโมดูลหลัก ๆ ดังนี้

- Video Loading และ Playing คือระบบจะให้ผู้ใช้งานเลือกไฟล์วิดีโอที่ต้องการวิเคราะห์ และเมื่อกดเล่น วิดีโอ ระบบจะเล่นพร้อมบันทึกทั้งหน้าจอและกล้องเพื่อการวิเคราะห์เพิ่มเติม
- Emotion Detection คือการใช้โมเดลการวิเคราะห์ใบหน้าและอารมณ์โดยใช้โมเดล trpakov/vit-face-expression จาก Hugging Face เพื่อตรวจจับใบหน้าและคำนวณความน่าจะเป็นของอารมณ์ที่ปรากฏ
- Data Visualization คือการแสดงผลการวิเคราะห์อารมณ์ในรูปแบบกราฟและแผนภูมิ

5.2.2 รายละเอียดการทำงานของระบบ

1) การโหลดและเล่นวิดีโอ (Video Loading and Playing)
   - เมื่อผู้ใช้งานเลือกไฟล์วิดีโอ ระบบจะทำการโหลดวิดีโอผ่าน tkinter.filedialog และเปิดการใช้งานปุ่มสำหรับเริ่มเล่นวิดีโอ (Play Video) รวมถึงปุ่มสำหรับเริ่มการวิเคราะห์ (Analyze Emotions)
   - เมื่อกดเริ่มเล่นวิดีโอ ระบบจะทำการเปิดเธรดเพื่อเล่นวิดีโอด้วย pygame ซึ่งจะเล่นวิดีโอพร้อมเสียง และเปิดกล้องสำหรับบันทึกเพื่อใช้ในการวิเคราะห์อารมณ์

2) การตรวจจับและวิเคราะห์อารมณ์ (Emotion Detection)
   - โมเดลการวิเคราะห์ใบหน้าและอารมณ์ใช้ไลบรารี facenet-pytorch สำหรับการตรวจจับใบหน้า (face detection) จากนั้นโมเดล ViT (Vision Transformer) จะทำการคำนวณความน่าจะเป็นของอารมณ์ต่าง ๆ เช่น happy, sad, angry
   - สำหรับแต่ละใบหน้าที่ตรวจจับได้ ระบบจะทำการแสดงผลความน่าจะเป็นของอารมณ์โดยสร้างกราฟแท่ง (bar chart),กราฟวงกลม (pie chart),คลิปวิดีโอการวิเคราะห์อารมณ์ (Emotion Analysis Video Clip) เพื่อแสดงอารมณ์ต่าง ๆ ที่ตรวจจับได้จากใบหน้าที่พบในแต่ละเฟรม

3) การสร้างวิดีโอผลลัพธ์ (Result Video Generation)
   - ระบบจะนำภาพของอารมณ์ที่ตรวจจับได้จากแต่ละเฟรมมาต่อกันเป็นคลิปวิดีโอด้วย ImageSequenceClip
   - ผลลัพธ์นี้จะถูกจัดเก็บและย้ายไปยังโฟลเดอร์ย่อยที่มีวันที่ปัจจุบัน

4) การแสดงผลกราฟและแผนภูมิ (Visualization)
   - ระบบจะสร้างแผนภูมิแสดงการกระจายตัวของอารมณ์ (emotion distribution) ในรูปแบบวงกลม (pie chart) และกราฟแท่ง (bar chart) แสดงความน่าจะเป็นของอารมณ์ต่าง ๆ ในแต่ละเฟรม เพื่อแสดงความเปลี่ยนแปลงของอารมณ์เมื่อเวลาผ่านไป
   - แผนภูมิและกราฟเหล่านี้จะถูกบันทึกเป็นไฟล์ PNG เพื่อใช้งานหรือแชร์ต่อ

5.2.3 ชุดคำสั่งและฟังก์ชันการทำงานของระบบ

มีการ pip install และ Library ต่าง ๆ ดังรูป

```
import cv2
import numpy as np
import os
import shutil
import torch
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from facenet_pytorch import MTCNN
from transformers import AutoFeatureExtractor, AutoModelForImageClassification, AutoConfig
from PIL import Image, ImageDraw
from moviepy.editor import VideoFileClip, ImageSequenceClip
from matplotlib.backends.backend_agg import FigureCanvasAgg as FigureCanvas
from tqdm import tqdm
import tkinter as tk
from tkinter import filedialog
import pyautogui
from threading import Thread
from datetime import datetime
import time
import matplotlib.patches as mpatches
import pygame
```

รูปที่ 5.6 คำสั่ง Library ที่นำมาใช้ทั้งหมด

โปรแกรมนี้นำเข้าไลบรารีสำคัญหลายตัว เช่น OpenCV สำหรับการประมวลผลภาพและวิดีโอ, NumPy สำหรับการจัดการอาร์เรย์, PyTorch สำหรับการพัฒนาและฝึกโมเดลการเรียนรู้เชิงลึก, และ Matplotlib สำหรับการสร้างกราฟและการแสดงผลข้อมูล โดยไลบรารีเหล่านี้มีบทบาทสำคัญในการตรวจจับใบหน้าและวิเคราะห์อารมณ์ของผู้ใช้จากวิดีโอเพื่อแสดงผลความน่าจะเป็นของอารมณ์ที่ตรวจจับได้ในรูปแบบกราฟและวิดีโอที่มีการบันทึกอารมณ์ที่ตรวจพบในแต่ละเฟรมอย่างมีประสิทธิภาพและแม่นยำ

```
class VideoPlayer:
    def __init__(self, root):
        self.root = root
        self.root.title("Video Player with Emotion Analysis")
        self.root.geometry(self.center_window(800, 600))
        self.vcap = None
        self.cam = None
        self.running = False
        self.recording = False
        self.video_path = None

        # Video load button
        self.btn_load = tk.Button(root, text="Load Video", command=self.load_video)
        self.btn_load.pack()

        # Play video and start recording button
        self.btn_play = tk.Button(root, text="Play Video", command=self.start_video_and_recording, state=tk.DISABLED)
        self.btn_play.pack()

        # Analyze emotions button
        self.btn_analyze = tk.Button(root, text="Analyze Emotions", command=self.analyze_emotions, state=tk.DISABLED)
        self.btn_analyze.pack()
```

รูปที่ 5.7 คลาส VideoPlayer

คลาส VideoPlayer ถูกออกแบบมาเพื่อสร้างอินเทอร์เฟซสำหรับการเล่นวิดีโอและวิเคราะห์อารมณ์ โดยเมธอด __init__ กำหนดค่าเริ่มต้น เช่น ชื่อและขนาดหน้าต่าง รวมถึงสถานะของวิดีโอและการบันทึก มีปุ่มสำหรับโหลดวิดีโอ เล่นวิดีโอ และวิเคราะห์อารมณ์ โดยปุ่มเหล่านี้จะถูกปิดการใช้งานจนกว่าจะมีการโหลดวิดีโอเข้าสู่ระบบ

ฟังก์ชัน center_window

ฟังก์ชันนี้ใช้สำหรับจัดตำแหน่งหน้าต่างโปรแกรมให้อยู่ตรงกลางหน้าจอ ฟังก์ชันจะคำนวณขนาดหน้าจอที่ใช้งานและตำแหน่งที่เหมาะสมสำหรับการแสดงผลโปรแกรม จากนั้นส่งค่าพิกัด x, y กลับมาเพื่อใช้ในการตั้งค่าตำแหน่งของหน้าต่างให้ปรากฏตรงกลางหน้าจอ เป็นการช่วยให้โปรแกรมดูเป็นระเบียบและผู้ใช้เห็นได้ง่าย

```
def center_window(self, width=300, height=200):
    screen_width = self.root.winfo_screenwidth()
    screen_height = self.root.winfo_screenheight()
    x = (screen_width // 2) - (width // 2)
    y = (screen_height // 2) - (height // 2)
    return f'{width}x{height}+{x}+{y}'
```

รูปที่ 5.8 คลาส VideoPlayer

ฟังก์ชัน load_video

ฟังก์ชันนี้ใช้เปิดหน้าต่างที่ให้ผู้ใช้เลือกไฟล์วิดีโอจากโฟลเดอร์เฉพาะ เมื่อผู้ใช้เลือกไฟล์วิดีโอแล้ว ฟังก์ชันจะบันทึกเส้นทางของไฟล์ที่เลือกไว้ในตัวแปร self.video_path และสร้างตัวจับวิดีโอ (video capture) โดยใช้ OpenCV ในตัวแปร self.vcap จากนั้นฟังก์ชันนี้ยังเปิดปุ่มสำหรับ "Play Video" และ "Analyze Emotions" เพื่อให้ผู้ใช้สามารถเล่นวิดีโอและเริ่มการวิเคราะห์อารมณ์ได้ โดยทำงานตามขั้นตอนต่อไปนี้

- ฟังก์ชันจะเปิดหน้าต่างให้ผู้ใช้เลือกไฟล์วิดีโอโดยใช้ filedialog.askopenfilename() ซึ่งจะช่วยให้ผู้ใช้สามารถเลือกไฟล์วิดีโอจากเครื่องคอมพิวเตอร์ที่ต้องการได้
- หลังจากเลือกไฟล์แล้ว ฟังก์ชันจะตรวจสอบนามสกุลของไฟล์ที่เลือก (เช่น .mp4, .avi) เพื่อให้แน่ใจว่าเป็นไฟล์วิดีโอที่รองรับ ถ้าไฟล์ไม่ใช่ประเภทที่ถูกต้อง ฟังก์ชันจะแสดงข้อความแจ้งเตือนถึงผู้ใช้ว่าให้เลือกไฟล์วิดีโอที่ถูกต้อง
- เมื่อผู้ใช้เลือกไฟล์ที่ถูกต้อง ฟังก์ชันจะเก็บเส้นทางของไฟล์นั้นลงในตัวแปร self.video_path, ซึ่งจะถูกใช้ในภายหลังเมื่อเริ่มการเล่นวิดีโอ
- ฟังก์ชันจะเปิดใช้งานปุ่ม Play Video และปุ่ม Analyze Emotions เพื่อให้ผู้ใช้สามารถเล่นวิดีโอและเริ่มการวิเคราะห์อารมณ์ได้ หลังจากที่มีการโหลดวิดีโอแล้ว

```
def load_video(self):
    self.video_path = filedialog.askopenfilename(initialdir="D:/Emotion Recognition and Analysis System/VDO")
    if self.video_path:
        self.vcap = cv2.VideoCapture(self.video_path)
        self.btn_play.config(state=tk.NORMAL)
        self.btn_analyze.config(state=tk.NORMAL)
```

รูปที่ 5.9 ฟังก์ชัน load_video

ฟังก์ชัน start_video_and_recording

ฟังก์ชันนี้ใช้สำหรับเริ่มการเล่นวิดีโอจากไฟล์ที่ผู้ใช้เลือก พร้อมกับเริ่มกระบวนการบันทึกและวิเคราะห์อารมณ์จากเฟรมวิดีโอที่กำลังเล่น โดยจะทำงานร่วมกับฟังก์ชันอื่น ๆ เพื่อให้สามารถตรวจจับและวิเคราะห์อารมณ์ในขณะที่วิดีโอกำลังแสดงอยู่โดยทำงานตามขั้นตอนดังนี้

- ฟังก์ชันเริ่มต้นด้วยการตรวจสอบว่าการบันทึกกำลังทำงานอยู่หรือไม่ หากยังไม่เริ่ม จะเปิดการบันทึก
- สร้างอ็อบเจ็กต์ VideoCapture เพื่ออ่านข้อมูลจากไฟล์วิดีโอที่ผู้ใช้เลือก เพื่อดึงเฟรมภาพจากวิดีโอ
- เริ่มลูปเพื่อดึงและแสดงเฟรมวิดีโออย่างต่อเนื่องในหน้าต่างโปรแกรม โดยใช้ฟังก์ชันการแสดงผลที่เหมาะสม
- ขณะเล่นวิดีโอ ฟังก์ชันจะเรียกใช้งานฟังก์ชันวิเคราะห์อารมณ์เพื่อตรวจสอบอารมณ์จากเฟรมที่ดึงขึ้นมา
- ตรวจสอบว่าผู้ใช้ต้องการหยุดการเล่นหรือไม่ หากต้องการ จะหยุดการเล่นและบันทึกข้อมูลที่ได้
- เมื่อเสร็จสิ้น ฟังก์ชันจะปิดการเชื่อมต่อกับวิดีโอและปล่อยทรัพยากรที่ใช้งาน เพื่อให้ระบบทำงานได้อย่างมีประสิทธิภาพ

```
def start_video_and_recording(self):
    if self.vcap is not None and not self.running:
        self.running = True
        self.recording = True
        self.cam = cv2.VideoCapture(0)

        video_thread = Thread(target=self.play_video)
        video_thread.start()

        record_thread = Thread(target=self.record_screen_and_cam)
        record_thread.start()
```

รูปที่ 5.10 ฟังก์ชัน start_video_and_recording

ฟังก์ชัน play_video

ฟังก์ชันนี้จะมีหน้าที่ในการเล่นวิดีโอที่ผู้ใช้เลือก โดยเริ่มจากการสร้างอ็อบเจ็กต์ VideoCapture เพื่อเปิดไฟล์วิดีโอ จากนั้นจะดึงและแสดงเฟรมวิดีโอในลูปอย่างต่อเนื่องจนกว่าวิดีโอจะสิ้นสุด เมื่อวิดีโอเล่นจนจบ ฟังก์ชันจะทำการปิดการเชื่อมต่อกับอ็อบเจ็กต์ VideoCapture และปล่อยทรัพยากรที่ใช้ทั้งหมดออก เพื่อให้ระบบทำงานได้อย่างมีประสิทธิภาพ

```
def play_video(self):
    if self.video_path is not None:

        pygame.init()
        pygame.mixer.init()

        clip = VideoFileClip(self.video_path)

        if clip:
            try:

                clip.preview()
            except Exception as e:
                print(f"Error during video preview: {e}")
            finally:

                self.stop_recording()
                clip.close()
                cv2.destroyAllWindows()
```

รูปที่ 5.11 ฟังก์ชัน play_video

ฟังก์ชัน record_screen_and_cam

ฟังก์ชันนี้ใช้ในการบันทึกวิดีโอจากกล้องเว็บแคมพร้อมกับการแสดงผลในหน้าต่าง GUI ของโปรแกรม โดยมีขั้นตอนการทำงานดังนี้

- ฟังก์ชันเริ่มต้นด้วยการสร้างอ็อบเจ็กต์ VideoCapture โดยใช้ cv2.VideoCapture(0) ซึ่งหมายเลข 0 จะบ่งบอกถึงกล้องหลักที่เชื่อมต่อกับคอมพิวเตอร์ เช่น กล้องในตัว (built-in camera) หรือกล้อง USB การสร้างอ็อบเจ็กต์นี้จะทำให้ฟังก์ชันสามารถดึงข้อมูลวิดีโอจากกล้องได้ในเวลาจริง
- ฟังก์ชันจะตั้งค่าขนาดของเฟรมที่ต้องการบันทึก โดยใช้ set() เพื่อกำหนดความกว้าง (width) และความสูง (height) ของเฟรม เช่น 640x480 ซึ่งเป็นขนาดที่ใช้บ่อยในวิดีโอการสื่อสาร
- ฟังก์ชันจะสร้างวิดีโอ codec โดยใช้ cv2.VideoWriter_fourcc(*'XVID') ซึ่งจะช่วยในการบีบอัดไฟล์วิดีโอให้มีขนาดเล็กลงในระหว่างการบันทึก
- จะมีการกำหนดชื่อไฟล์ที่บันทึก (เช่น output.avi), codec, อัตราเฟรมต่อวินาที (fps) เช่น 20.0, และขนาดของเฟรม (width, height) เพื่อเริ่มต้นการบันทึกวิดีโอ
- ฟังก์ชันจะเริ่มต้นลูป while เพื่อดึงเฟรมจากกล้อง โดยใช้ self.cam.read() ซึ่งจะส่งกลับสองค่า: ตัวแปร ret ที่บอกสถานะการอ่านเฟรมว่าประสบความสำเร็จหรือไม่ และตัวแปร frame ที่เก็บข้อมูลเฟรมที่ดึงมาจากกล้อง หากการอ่านเฟรมสำเร็จ ฟังก์ชันจะทำการแสดงเฟรมนั้นในหน้าต่าง GUI โดยใช้ cv2.imshow() เพื่อให้ผู้ใช้เห็นภาพที่ถูกบันทึก
- นอกจากการแสดงผลเฟรมในหน้าต่าง GUI แล้ว ฟังก์ชันจะทำการเขียนเฟรมที่ดึงมาได้ลงในไฟล์วิดีโอที่ถูกสร้างขึ้นก่อนหน้านี้ โดยใช้ self.out.write(frame)
- หลังจากหยุดการบันทึก ฟังก์ชันจะทำการเรียก self.cam.release() เพื่อปิดการเชื่อมต่อกับกล้อง และ self.out.release() เพื่อปิดการเชื่อมต่อกับอ็อบเจ็กต์ VideoWriter
- นอกจากนี้ ฟังก์ชันจะใช้ cv2.destroyAllWindows() เพื่อปิดหน้าต่างที่แสดงวิดีโอที่เปิดอยู่ ทำให้ระบบสามารถปล่อยทรัพยากรที่ใช้งานไปแล้วกลับสู่ระบบได

```
def record_screen_and_cam(self):
    screen_size = pyautogui.size()
    fourcc = cv2.VideoWriter_fourcc(*"XVID")
    out_screen = cv2.VideoWriter('screen_output.avi', fourcc, 20.0, (screen_size.width, screen_size.height))
    out_cam = cv2.VideoWriter('cam_output.avi', fourcc, 30.0, (640, 480))

    while self.recording:
        img = pyautogui.screenshot()
        screen_frame = np.array(img)
        screen_frame = cv2.cvtColor(screen_frame, cv2.COLOR_BGR2RGB)
        out_screen.write(screen_frame)

        ret, cam_frame = self.cam.read()
        if ret:
            out_cam.write(cam_frame)

    out_screen.release()
    out_cam.release()
    self.cam.release()
```

รูปที่ 5.12 ฟังก์ชัน record_screen_and_cam

ฟังก์ชัน stop_recording

ฟังก์ชันนี้มีหน้าที่ในการหยุดกระบวนการบันทึกวิดีโอจากกล้องเมื่อมีการเรียกใช้งาน โดยเริ่มต้นจากการตรวจสอบสถานะของการบันทึก หากมีการบันทึกกำลังดำเนินการอยู่ จะทำการตั้งค่า self.recording เป็น False เพื่อตอกย้ำว่าการบันทึกได้สิ้นสุดลง จากนั้นจะปล่อยการเชื่อมต่อกับกล้องผ่าน self.cam.release()

```
def stop_recording(self):
    self.recording = False
    self.running = False
```

รูปที่ 5.13 ฟังก์ชัน stop_recording

ฟังก์ชัน analyze_emotions

ฟังก์ชันนี้มีหน้าที่วิเคราะห์อารมณ์ของผู้ใช้จากเฟรมวิดีโอที่กำลังเล่นอยู่ โดยมีขั้นตอนการทำงานที่สำคัญดังนี้

- ฟังก์ชันเริ่มต้นด้วยการตรวจสอบว่าตัวแปร self.vcap (ซึ่งอาจใช้ในการตรวจจับวิดีโอ) มีค่าเป็น None หรือไม่ ซึ่งหมายความว่ามีวิดีโอที่โหลดไว้ หากไม่มีวิดีโอ ฟังก์ชันจะไม่ทำงานต่อไป
- ตัวแปร video_path จะเปิดหน้าต่างเพื่อให้ผู้ใช้เลือกไฟล์วิดีโอจากเครื่อง โดยเริ่มต้นที่โฟลเดอร์ที่ได้กำหนด (D:/Emotion Recognition and Analysis System) และเก็บเส้นทางของไฟล์วิดีโอที่เลือกไว้ในตัวแปร
- ต่อไปจะเรียกใช้ process_video() โดยส่งเส้นทางของวิดีโอที่ผู้ใช้เลือกไป ซึ่งฟังก์ชันนี้จะประมวลผลวิดีโอและคืนค่าพาธของวิดีโอที่ถูกประมวลผลและความน่าจะเป็นของอารมณ์ที่ตรวจจับได้ในรูปแบบของอาเรย์ไว้ในตัวแปร output_video_path และ all_class_probabilities ตามลำดับ
- จะเรียกใช้ plot_emotion_distribution() เพื่อสร้างกราฟแสดงการแจกแจงอารมณ์จากค่าความน่าจะเป็นที่ได้จากการประมวลผลวิดีโอ
- สุดท้ายจะเรียกใช้ move_output_files() เพื่อย้ายไฟล์วิดีโอที่ถูกประมวลผลไปยังโฟลเดอร์ 'output' โดยใช้พาธของไฟล์วิดีโอที่ได้เป็นพารามิเตอร์

```
def analyze_emotions(self):
    if self.vcap is not None:
        video_path = filedialog.askopenfilename(initialdir="D:/Emotion Recognition and Analysis System")
        output_video_path, all_class_probabilities = process_video(video_path)
        plot_emotion_distribution(all_class_probabilities)
        move_output_files(output_video_path, 'output')
```

รูปที่ 5.14 ฟังก์ชัน analyze_emotions

ฟังก์ชัน detect_emotions(image, mtcnn, extractor, model)

ฟังก์ชันนี้ทำงานเพื่อตรวจจับและวิเคราะห์อารมณ์จากภาพใบหน้า โดยมีขั้นตอนดังนี้

- ใช้ MTCNN ในการค้นหากรอบใบหน้าในภาพ เพื่อให้ได้พิกัดของใบหน้า โดยผลลัพธ์จะเป็นลิสต์ที่ประกอบไปด้วยข้อมูลเกี่ยวกับใบหน้าและกรอบที่ตรวจจับได้
- ตรวจสอบว่ามีใบหน้าถูกตรวจจับหรือไม่ โดยเช็คว่าลิสต์แรกในผลลัพธ์ไม่ใช่ None
- ดึงกรอบใบหน้าที่ถูกตรวจจับมาเก็บในตัวแปร box ซึ่งจะเป็นพิกัดของกรอบ (x_min, y_min, x_max, y_max) และ ตัดส่วนของภาพที่อยู่ภายในกรอบใบหน้าที่ได้จากตัวแปร box โดยใช้ฟังก์ชัน crop
- ประมวลผลด้วย extractor เพื่อแปลงเป็นฟีเจอร์ในรูปแบบ tensors ที่เหมาะสมสำหรับการป้อนเข้าสู่โมเดล จากนั้นโมเดลจะทำการคาดการณ์อารมณ์ โดยใช้ฟังก์ชัน Softmax เพื่อแปลงผลลัพธ์ให้เป็นความน่าจะเป็นของแต่ละอารมณ์
- โมเดลจะโหลดการตั้งค่าจากการฝึกฝนล่วงหน้าเพื่อให้ได้ข้อมูลการแมพป้ายชื่ออารมณ์
- สุดท้ายจะสร้างดิกชันนารีที่เก็บค่าความน่าจะเป็นของแต่ละอารมณ์และคืนค่าภาพใบหน้าพร้อมกับข้อมูลนี้กลับไปยังผู้เรียกใช้งาน หากไม่พบใบหน้า ฟังก์ชันจะคืนค่า None ทั้งสองค่า

```python
def detect_emotions(image, mtcnn, extractor, model):
    temporary = image.copy()

    sample = mtcnn.detect(temporary)
    if sample[0] is not None:
        box = sample[0][0]

        face = temporary.crop(box)

        inputs = extractor(images=face, return_tensors="pt")

        outputs = model(**inputs)

        probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)

        config = AutoConfig.from_pretrained("trpakov/vit-face-expression")
        id2label = config.id2label

        probabilities = probabilities.detach().numpy().tolist()[0]

        class_probabilities = {id2label[i]: prob for i, prob in enumerate(probabilities)}

        return face, class_probabilities
    return None, None
```

รูปที่ 5.15 ฟังก์ชัน detect_emotions

ฟังก์ชัน create_combined_image(face, class_probabilities)

ฟังก์ชันนี้จะสร้างภาพที่รวมภาพใบหน้าพร้อมกับข้อมูลอารมณ์ที่ตรวจจับได้ โดยจะทำงานตามขั้นตอนดังนี้

- ฟังก์ชันรับพารามิเตอร์สองตัว ได้แก่ face ซึ่งเป็นภาพใบหน้าที่ถูกตัดออกมา และ class_probabilities ซึ่งเป็นดิกชันนารีที่ประกอบด้วยชื่ออารมณ์และค่าความน่าจะเป็นที่ตรวจจับได้
- ใช้ Image.new() จากไลบรารี PIL (Python Imaging Library) เพื่อสร้างภาพใหม่ที่มีสีพื้นหลังเป็นสีขาว (RGB: 255, 255, 255) และมีขนาดตามที่คำนวณไว้
- ใช้ฟังก์ชัน paste() เพื่อวางภาพใบหน้าลงในภาพพื้นหลังที่สร้างขึ้น โดยกำหนดตำแหน่งที่ต้องการวาง (ในที่นี้คือพิกัด (100, 50))
- สร้างข้อความที่จะวางบนภาพโดยการสร้างสตริงที่รวมชื่ออารมณ์และค่าความน่าจะเป็น โดยใช้การวนลูปใน class_probabilities และจัดรูปแบบค่าให้มีทศนิยมสองตำแหน่ง
- ใช้ ImageDraw.Draw() เพื่อสร้างวัตถุที่สามารถวาดข้อความลงบนภาพ
- กำหนดฟอนต์ที่จะใช้สำหรับวาดข้อความ โดยในที่นี้ใช้ฟอนต์เริ่มต้น
- สุดท้ายจะคืนค่าภาพที่มีทั้งใบหน้าและข้อมูลอารมณ์รวมอยู่ในภาพเดียว เพื่อให้สามารถนำไปแสดงผลหรือใช้งานต่อไปได้

```python
def create_combined_image(face, class_probabilities):
    colors = {
        "angry": "red",
        "disgust": "green",
        "fear": "gray",
        "happy": "yellow",
        "neutral": "purple",
        "sad": "blue",
        "surprise": "orange"
    }
    palette = [colors[label] for label in class_probabilities.keys()]

    fig, axs = plt.subplots(1, 2, figsize=(15, 6))

    axs[0].imshow(np.array(face))
    axs[0].axis('off')

    sns.barplot(ax=axs[1],
                y=list(class_probabilities.keys()),
                x=[prob * 100 for prob in class_probabilities.values()],
                palette=palette,
                orient='h')
    axs[1].set_xlabel('Probability (%)')
    axs[1].set_title('Emotion Probabilities')
    axs[1].set_xlim([0, 100])

    canvas = FigureCanvas(fig)
    canvas.draw()
    img = np.frombuffer(canvas.tostring_rgb(), dtype='uint8')
    img = img.reshape(canvas.get_width_height()[::-1] + (3,))
    plt.close(fig)
    return img
```

รูปที่ 5.16 ฟังก์ชัน create_combined_image

ฟังก์ชัน process_video

- ตรวจสอบว่า GPU สามารถใช้งานได้หรือไม่ หากใช้งานได้จะเลือกใช้ GPU (cuda) มิฉะนั้นจะใช้ CPU (cpu)
- สร้างอ็อบเจ็กต์ MTCNN เพื่อทำการตรวจจับใบหน้าในภาพ โดยมีการกำหนดค่าต่าง ๆ เช่น
  - image_size =160: ขนาดภาพที่ใช้ในการตรวจจับใบหน้า
  - min_face_size =200: ขนาดใบหน้าขั้นต่ำที่ต้องการให้ตรวจจับ
  - thresholds: ค่าความเชื่อมั่นในการตรวจจับใบหน้า
  - device: ระบุว่าให้ใช้ CPU หรือ GPU ในการประมวลผล
- ใช้ AutoFeatureExtractor เพื่อเตรียมการประมวลผลฟีเจอร์ของภาพใบหน้าก่อนนำเข้าสู่โมเดล
- ใช้ AutoModelForImageClassification เพื่อโหลดโมเดลการจำแนกอารมณ์ที่ได้รับการฝึกไว้ล่วงหน้า ซึ่งจะใช้ในการคาดการณ์อารมณ์จากฟีเจอร์ที่ถูกดึงออกมาจากภาพใบหน้า

```
def process_video(video_path, frame_skip = 2 ):

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Using device: {device}")
    mtcnn = MTCNN(
        image_size=160,
        margin=0,
        min_face_size=200,
        thresholds=[0.6, 0.7, 0.7],
        factor=0,
        post_process=True,
        keep_all=False,
        device=device
    )

    extractor = AutoFeatureExtractor.from_pretrained("trpakov/vit-face-expression")
    model = AutoModelForImageClassification.from_pretrained("trpakov/vit-face-expression")
```

รูปที่ 5.17 ฟังก์ชัน process_video (1)

สร้างอ็อบเจ็กต์ VideoFileClip เพื่ออ่านวิดีโอและเก็บอัตราเฟรมต่อวินาที (FPS) ในตัวแปร vid_fps ข้อมูลเฟรมจะถูกแปลงเป็นอาร์เรย์ NumPy และประกาศรายการอารมณ์ที่ใช้ตรวจจับ พร้อมสร้างลิสต์ combined_images และ all_class_probabilities เพื่อเก็บผลการวิเคราะห์อารมณ์

```
clip = VideoFileClip(video_path)
vid_fps = clip.fps
video_data = np.array(list(clip.iter_frames()))

emotions = ["angry", "disgust", "fear", "happy", "neutral", "sad", "surprise"]

combined_images = []
all_class_probabilities = []

start_time_total = time.time()
skipped_frames = 0
processed_frames = 0
```

รูปที่ 5.18 ฟังก์ชัน process_video (2)

ส่วนนี้จะทำการวนลูปผ่านเฟรมวิดีโอ โดยใช้ tqdm เพื่อแสดงความก้าวหน้าในการประมวลผลเฟรมแต่ละเฟรม ถ้าเฟรมที่ประมวลผลไม่ตรงตาม frame_skip จะข้ามไป และนับจำนวนเฟรมที่ข้ามไป หลังจากนั้นแปลงเฟรมเป็นชนิดข้อมูล uint8 และเรียกใช้ฟังก์ชัน detect_emotions เพื่อตรวจจับอารมณ์ หากพบใบหน้า จะสร้างภาพรวมอารมณ์และเก็บไว้ใน combined_images มิฉะนั้นจะเก็บความน่าจะเป็นเป็น None สำหรับอารมณ์ต่าง ๆ สุดท้ายแสดงจำนวนเฟรมที่ประมวลผลและข้ามไป พร้อมทั้งเวลาที่ใช้ในการประมวลผลวิดีโอ หลังจากนั้นจะบันทึกภาพรวมเป็นวิดีโอใหม่ และส่งคืน output_video_path และ all_class_probabilities ซึ่งเป็นข้อมูลความน่าจะเป็นของอารมณ์ที่ตรวจจับได้

```
for i, frame in tqdm(enumerate(video_data), total=len(video_data), desc="Processing frames"):
    if i % frame_skip != 0:
        skipped_frames += 1
        continue  # Skip frames according to frame_skip

    frame = frame.astype(np.uint8)

    face, class_probabilities = detect_emotions(Image.fromarray(frame), mtcnn, extractor, model)

    if face is not None:
        combined_image = create_combined_image(face, class_probabilities)
        combined_images.append(combined_image)
    else:
        class_probabilities = {emotion: None for emotion in emotions}

    all_class_probabilities.append(class_probabilities)
    processed_frames += 1

end_time_total = time.time()  # End timing the whole process
print(f"Processed {processed_frames} frames. Skipped {skipped_frames} frames.")
print(f"Time taken for processing video: {end_time_total - start_time_total:.2f} seconds")

# Save the combined images as a video
output_video_path = video_path.replace('.mp4', '_emotion_analysis.mp4')
image_clip = ImageSequenceClip(combined_images, fps=vid_fps)

# Check if the original clip has audio before trying to set it
if clip.audio is not None:
    image_clip = image_clip.set_audio(clip.audio)

try:
    # Write video file with codec specified
    image_clip.write_videofile(output_video_path, codec='libx264', audio_codec='aac')
except Exception as e:
    print(f"Error writing video file: {e}")

return output_video_path, all_class_probabilities
```

รูปที่ 5.19 ฟังก์ชัน process_video (3)

ฟังก์ชัน plot_emotion_distribution(all_class_probabilities)

ฟังก์ชันนี้สร้างกราฟแสดงการแจกแจงอารมณ์จากข้อมูลความน่าจะเป็น โดยกำหนดสีให้กับแต่ละอารมณ์ในพจนานุกรม colors จากนั้นสร้าง DataFrame จาก all_class_probabilities, คูณค่าความน่าจะเป็นด้วย 100 เพื่อแสดงเป็นเปอร์เซ็นต์ และรวมความน่าจะเป็นทั้งหมดด้วย df.sum() เพื่อเตรียมข้อมูลสำหรับการแสดงผลกราฟ

```
def plot_emotion_distribution(all_class_probabilities):
    colors = {
        "angry": "red",
        "disgust": "green",
        "fear": "gray",
        "happy": "yellow",
        "neutral": "purple",
        "sad": "blue",
        "surprise": "orange"
    }

    df = pd.DataFrame(all_class_probabilities)
    df = df * 100
    total_probabilities = df.sum()
```

รูปที่ 5.20 ฟังก์ชัน plot_emotion_distribution (1)

จะเรียงลำดับค่าตามความน่าจะเป็นในลำดับจากมากไปหาน้อย จากนั้นสร้างกราฟพายโดยใช้ plt.subplots() เพื่อเตรียมกราฟสองกราฟในรูปแบบขนาดที่กำหนด โดยกราฟแรกจะแสดงการแจกแจงของอารมณ์ในรูปแบบกราฟพายที่มีขนาดใหญ่ขึ้น มีการกำหนดสีจากพจนานุกรม colors, แสดงเปอร์เซ็นต์ด้วย autopct, และตั้งชื่อกราฟเป็น "Emotion Probabilities Distribution" โดยยังมีการกำหนดให้กราฟพายเป็นวงกลม ด้วย ax1.set_aspect('equal') ขณะที่กราฟที่สองจะถูกตั้งค่าให้ไม่แสดงข้อมูลใด ๆ และยังคงรักษาสัดส่วนเป็นวงกลมด้วย ax2.set_aspect('equal') และคำนวณตำแหน่งแนวตั้ง (y_positions) สำหรับข้อความที่จะแสดงในกราฟ

```
percentages = total_probabilities / total_probabilities.sum() * 100
percentages = percentages.sort_values(ascending=False)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))

wedges, texts, autotexts = ax1.pie(percentages,
                                   colors=[colors[emotion] for emotion in percentages.index],
                                   autopct='%1.1f%%',
                                   startangle=90,
                                   radius=1.2)
ax1.set_title('Emotion Probabilities Distribution')
ax1.set_aspect('equal')
ax2.axis('off')
ax2.set_aspect('equal')
y_positions = np.linspace(0.9, 0.1, len(percentages))
```

รูปที่ 5.21 ฟังก์ชัน plot_emotion_distribution (2)

ในส่วนนี้ของฟังก์ชัน จะวนลูปผ่านค่าความน่าจะเป็นของอารมณ์และตำแหน่งแนวตั้งเพื่อสร้างวงกลมแสดงอารมณ์แต่ละอันในกราฟที่สอง พร้อมแสดงชื่ออารมณ์และเปอร์เซ็นต์ที่เกี่ยวข้อง โดยใช้ mpatches.Circle เพื่อเพิ่มวงกลมสี และ ax2.text เพื่อแสดงข้อความ จากนั้นปรับระยะห่างระหว่างกราฟด้วย plt.subplots_adjust(wspace=0.3) และบันทึกกราฟพายเป็นไฟล์ emotion_pie_chart.png ก่อนที่จะสร้างกราฟแท่งเพื่อแสดงความน่าจะเป็นของอารมณ์ตลอดเวลา โดยกำหนดแกน X และ Y พร้อมตั้งชื่อกราฟ และบันทึกเป็นไฟล์ emotion_bar_plot.png

```
for (emotion, percentage), y_pos in zip(percentages.items(), y_positions):
    circle = mpatches.Circle((0.05, y_pos), 0.03, color=colors[emotion], transform=ax2.transAxes, clip_on=False)
    ax2.add_patch(circle)
    ax2.text(0.1, y_pos, f"{emotion}: {percentage:.1f}%", transform=ax2.transAxes, fontsize=12, verticalalignment='center')

plt.subplots_adjust(wspace=0.3)
plt.savefig('emotion_pie_chart.png')

plt.figure(figsize=(15, 8))
for emotion in df.columns:
    plt.bar(df.index, df[emotion], label=emotion, color=colors[emotion])

plt.xlabel('Frame Order')
plt.ylabel('Emotion Probability (%)')
plt.title('Emotion Probabilities Over Time')
plt.legend(loc='upper right')
plt.savefig('emotion_bar_plot.png')
```

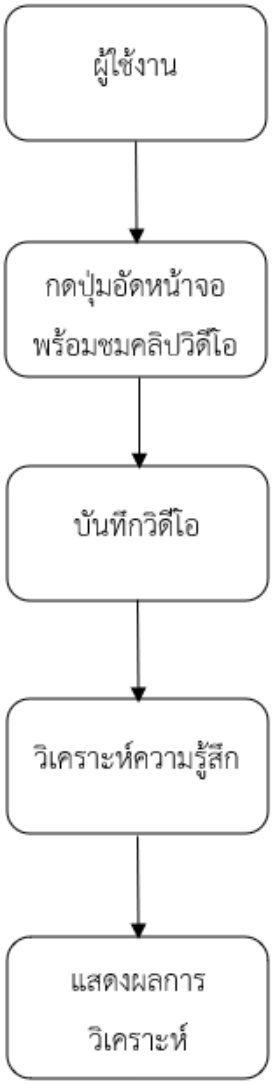
รูปที่ 5.22 ฟังก์ชัน plot_emotion_distribution (3)

ในฟังก์ชันนี้ จะตรวจสอบว่ามีโฟลเดอร์สำหรับเก็บไฟล์ผลลัพธ์หรือไม่ หากไม่มีจะสร้างขึ้นมา จากนั้นสร้างโฟลเดอร์ย่อยตามวันที่ปัจจุบัน และหากมีโฟลเดอร์ที่มีชื่อซ้ำ จะเพิ่มหมายเลขต่อท้ายเพื่อสร้างชื่อใหม่ หลังจากนั้นจะสร้างชื่อไฟล์ใหม่สำหรับไฟล์วิดีโอผลลัพธ์ โดยตรวจสอบว่าชื่อไฟล์นั้นมีอยู่แล้วหรือไม่ ก่อนที่จะทำการย้ายไฟล์วิดีโอและไฟล์กราฟพายและกราฟแท่งไปยังโฟลเดอร์ที่สร้างขึ้น และแสดงข้อความยืนยันการบันทึกไฟล์

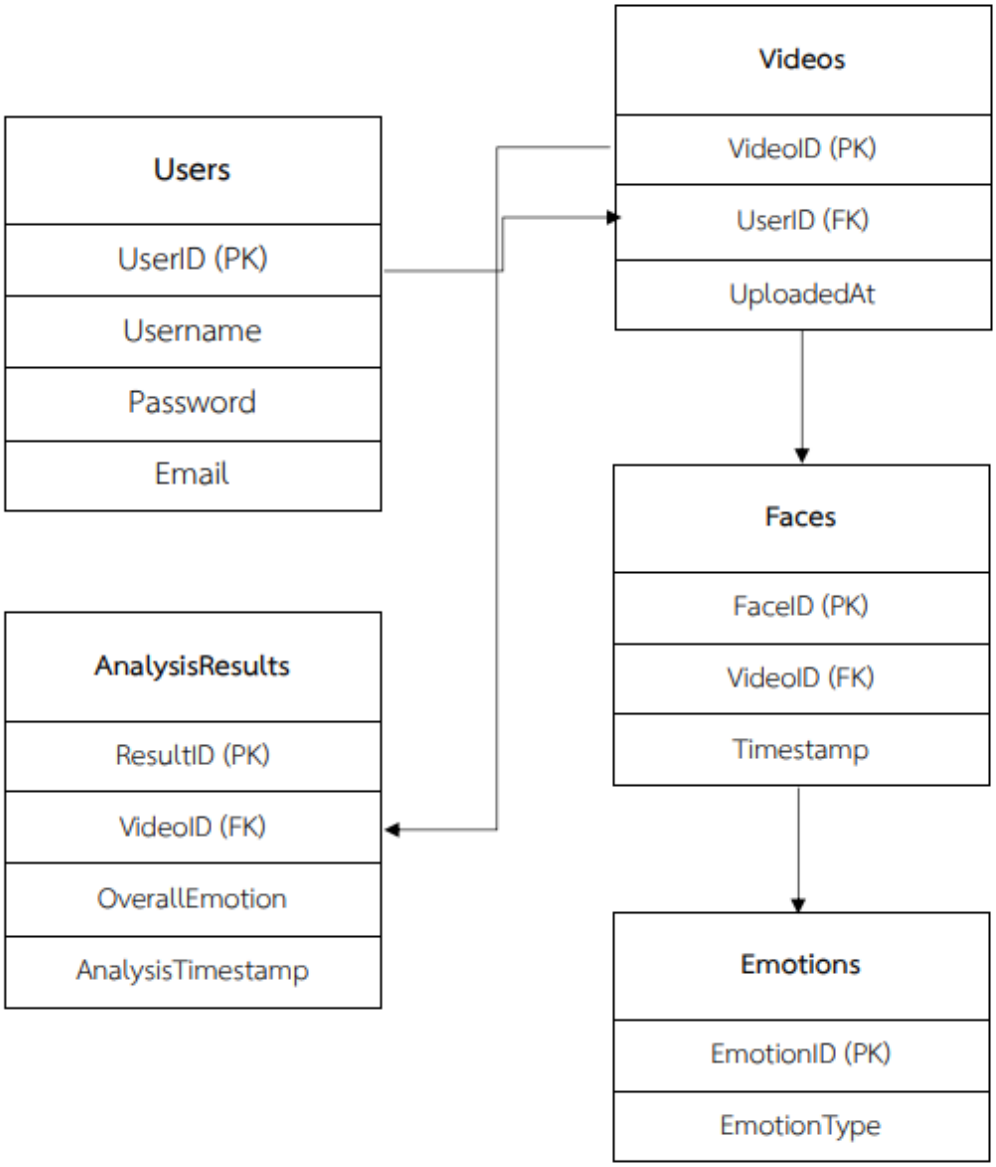
```
def move_output_files(output_video_path, output_directory):
    if not os.path.exists(output_directory):
        os.makedirs(output_directory)

    # สร้างโฟลเดอร์ย่อยตามวันที่ปัจจุบัน
    today = datetime.now().strftime('%d-%m-%Y')
    date_directory = os.path.join(output_directory, today)

    counter = 1
    while os.path.exists(date_directory):
        date_directory = os.path.join(output_directory, f"{today}_{counter}")
        counter += 1

    os.makedirs(date_directory)

    base_filename = os.path.splitext(os.path.basename(output_video_path))[0]
    output_video_ext = os.path.splitext(output_video_path)[1]

    counter = 1

    while True:
        new_filename = f"{base_filename}_{counter}{output_video_ext}"
        new_filepath = os.path.join(date_directory, new_filename)
        if not os.path.exists(new_filepath):
            break
        counter += 1

    shutil.move(output_video_path, new_filepath)
    shutil.move('emotion_pie_chart.png', os.path.join(date_directory, f'emotion_pie_chart.png'))
    shutil.move('emotion_bar_plot.png', os.path.join(date_directory, f'emotion_bar_plot.png'))

    print(f"Output files saved in the '{date_directory}' directory.")
```

รูปที่ 5.23 ฟังก์ชัน plot_emotion_distribution (4)

5.3 กระบวนการทำงานรูปแบบแผนผัง Block Diagram



รูปที่ 5.24 กระบวนการทำงานรูปแบบแผนผัง Block Diagram (1)



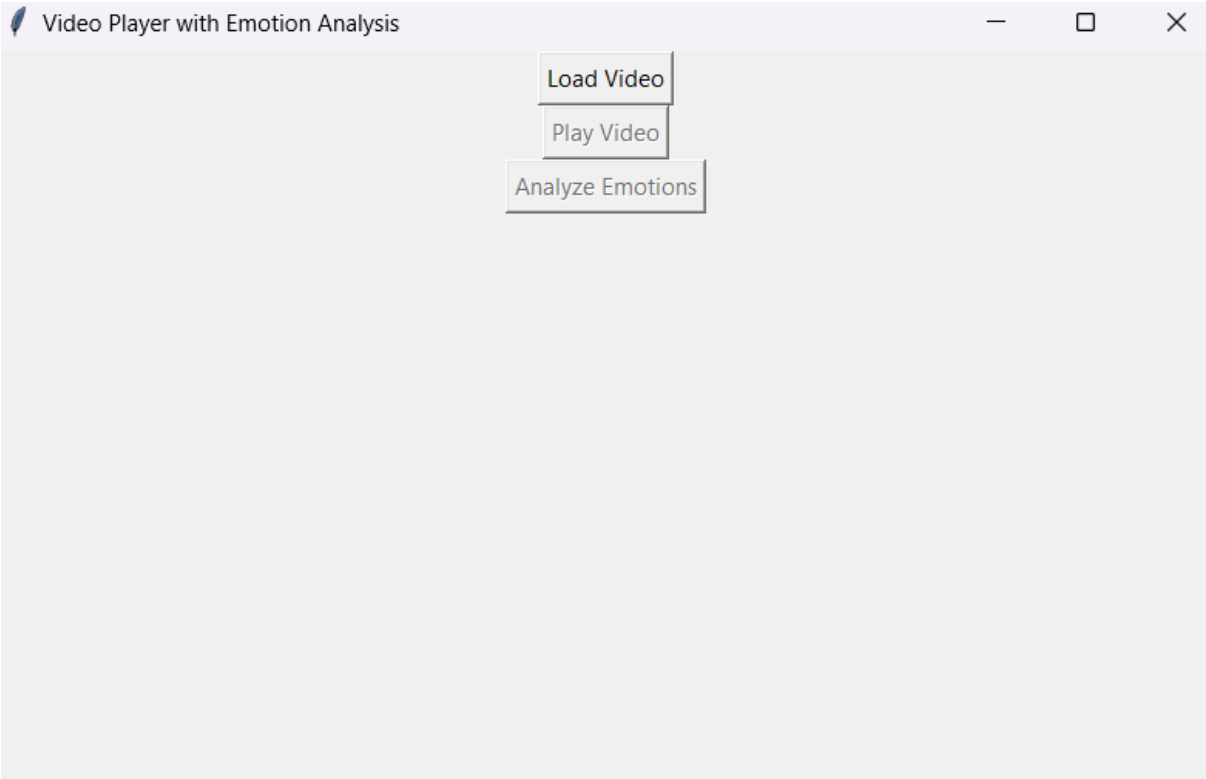
รูปที่ 5.25 กระบวนการทำงานรูปแบบแผนผัง Block Diagram (2)

บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

6.1 ผลการดำเนินงาน

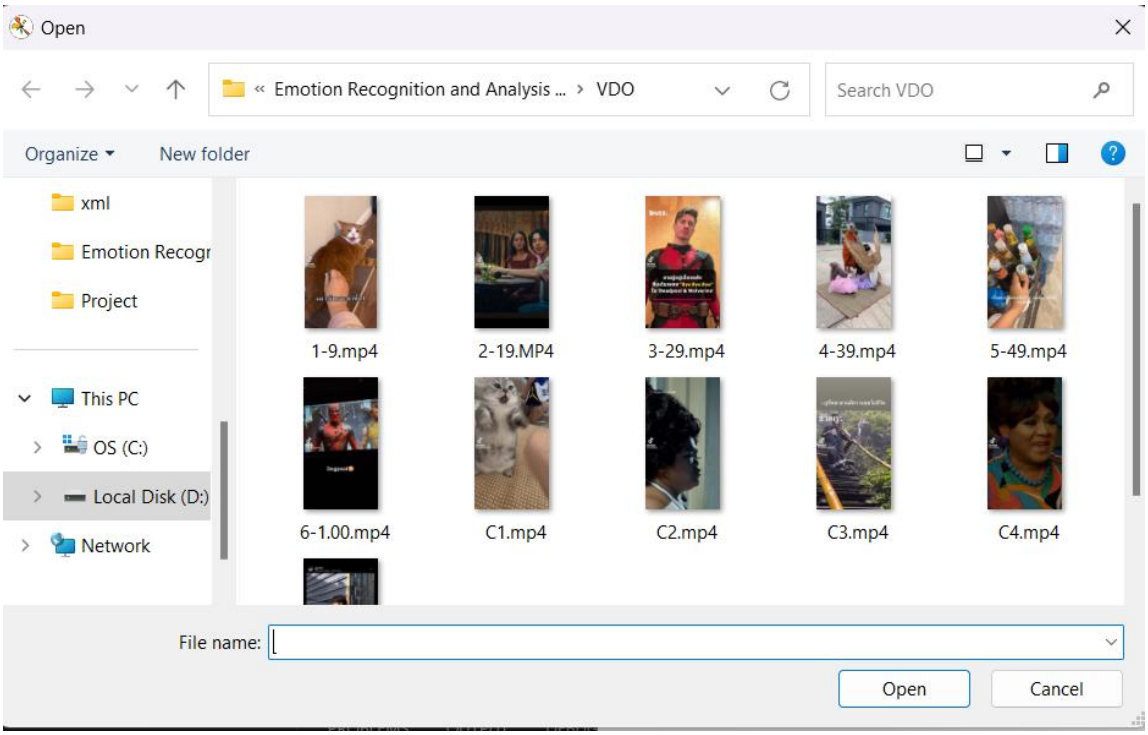
1) สามารถเปิดโปรแกรมผ่าน Visual Studio Code

- ปุ่มโหลด Load Video คือ การเลือกวิดีโอที่ต้องการรับชม
- ปุ่ม Play Video คือ การเล่นวิดีโอที่เลือก
- ปุ่ม Analyze Emotions คือ การวิเคราะห์อารมณ์จากการอัดหน้าจอขณะรับชม



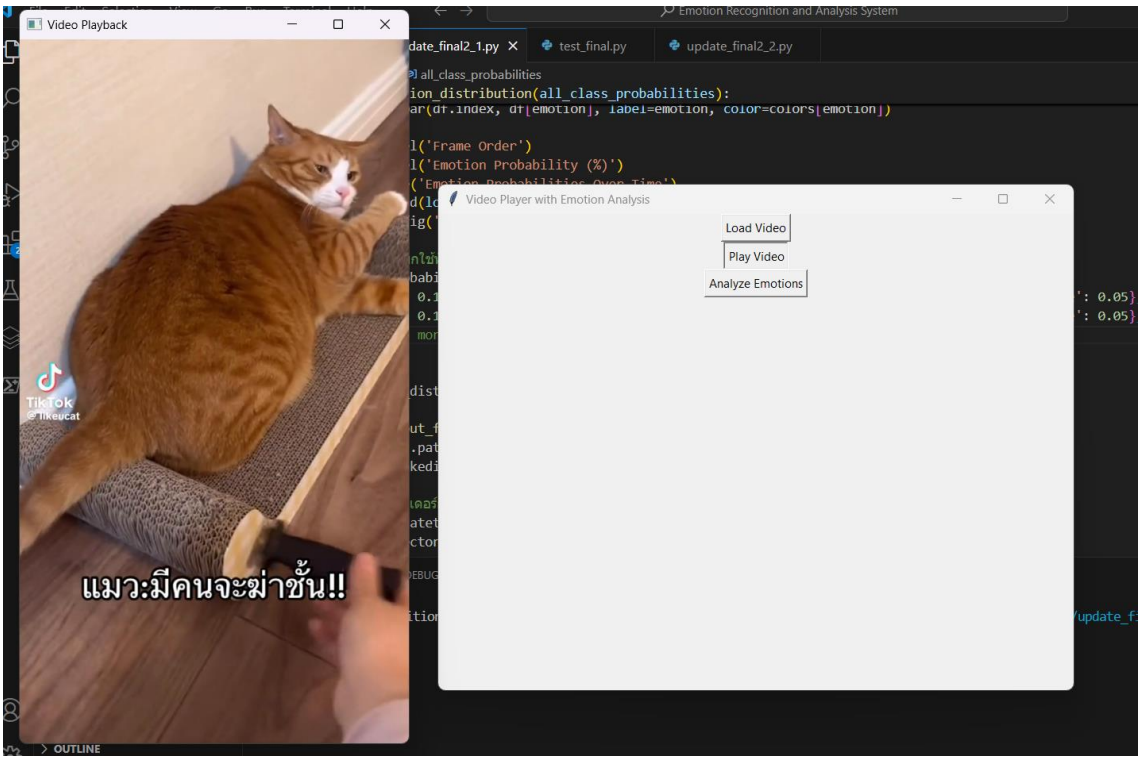
รูปที่ 6.1 โปรแกรม Video Player with Emotion Analysis

2) กดปุ่ม Load Video ที่หน้าต่างโปรแกรม และเลือกไฟล์วิดีโอที่ต้องการ



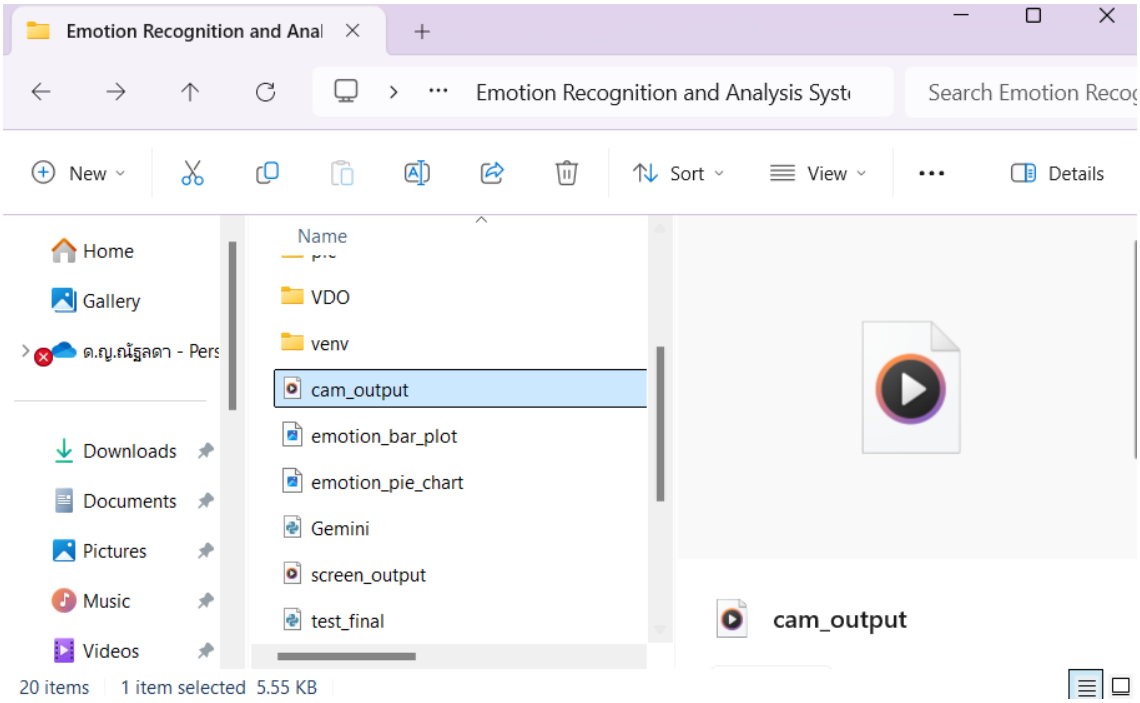
รูปที่ 6.2 โฟลเดอร์ที่เก็บวิดีโอ

3) เมื่อเลือกวิดีโอที่ต้องการแล้ว กดปุ่ม Play Video ระบบจะทำการอัดหน้าจอในขณะที่รับชมวิดีโอ ตั้งแต่ต้นจนจบคลิปวิดีโอ



รูปที่ 6.3 แสดงวิดีโอที่เลือก

4) เมื่อรับชมเสร็จเรียบร้อย กดปุ่ม Analyze Emotions โดยเข้ามาเลือกไฟล์ cam_output ในโฟลเดอร์ เพื่อทำการวิเคราะห์อารมณ์จากใบหน้า



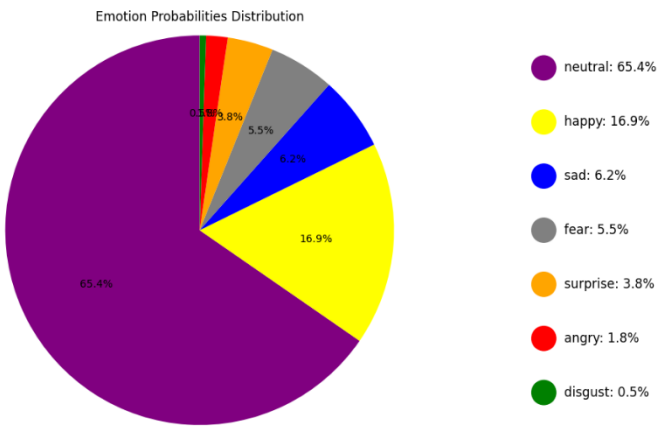
รูปที่ 6.4 เลือกวิดีโอที่อัดหน้าจอของผู้ใช้

5) รอการประมวลผล

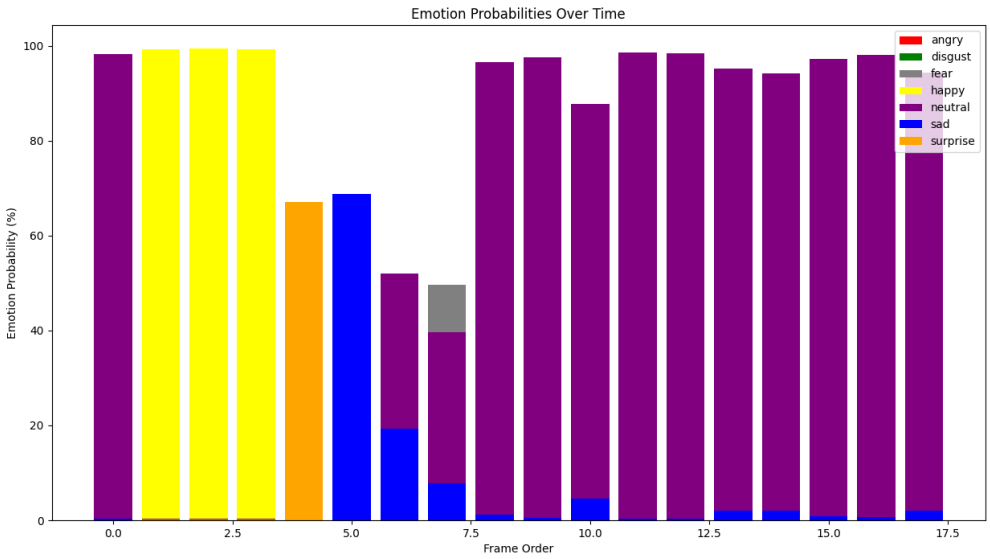
```
Processing frames: 100%|                                                    | 220/220 [00:43<00:00,  5.09it/s]
Total processing time: 0 minutes 43.22 seconds
Skipped frames: 176
Processed frames: 44
Moviepy - Building video output_video.mp4.
Moviepy - Writing video output_video.mp4
```

รูปที่ 6.5 การประมวลผลวิดีโอ

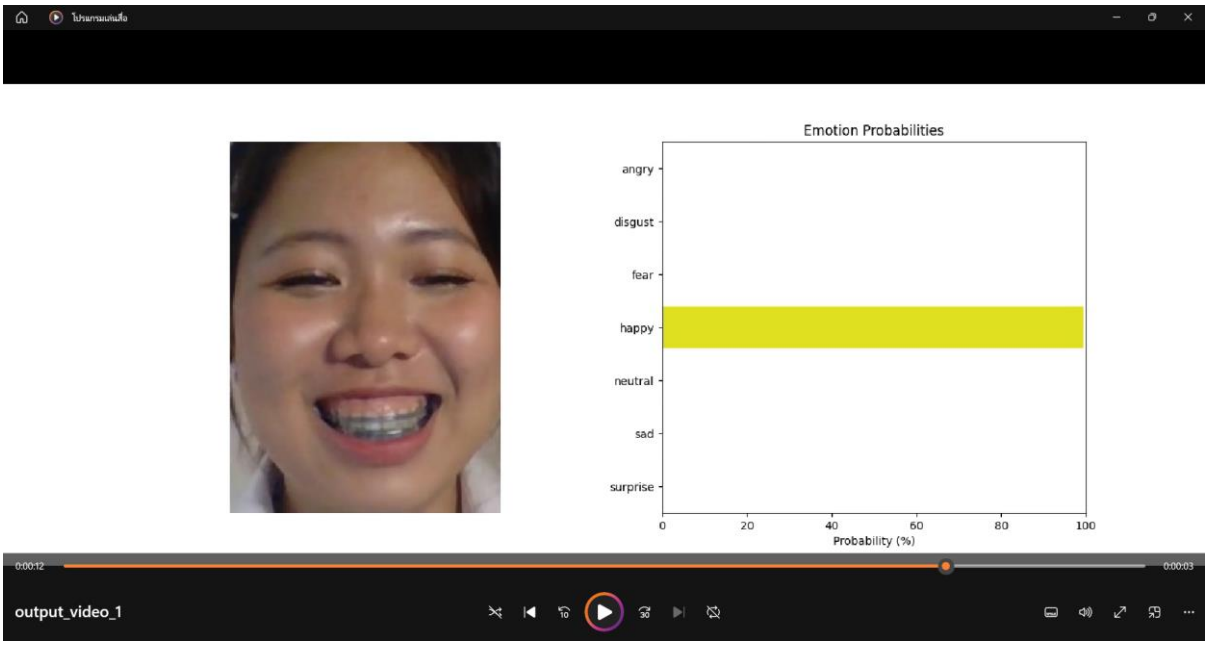
6) แสดงผลลัพธ์ออกมาดังนี้



รูปที่ 6.6 กราฟวงกลม (Pie Chart)



รูปที่ 6.7 กราฟแท่ง (Bar Chart)



รูปที่ 6.8 ผลความน่าจะเป็นของอารมณ์ต่าง ๆ

6.2 ข้อจำกัด

1) ข้อจำกัดของโปรแกรมนี้อยู่ที่ความแตกต่างของสภาพแวดล้อมและคุณลักษณะของคอมพิวเตอร์ที่ใช้งาน ซึ่งอาจส่งผลให้ประสิทธิภาพของการทำงานไม่คงที่หรือแปรผันไปตามอุปกรณ์
2) การตรวจจับและวิเคราะห์อารมณ์จากวิดีโออาจมีความแม่นยำต่ำในบางกรณี เช่น เมื่อมีการปรับแสงที่ไม่เหมาะสม หรือใบหน้าของบุคคลไม่ชัดเจน
3) อาจมีปัญหาในการวิเคราะห์อารมณ์เมื่อมีใบหน้าหลายใบหน้าในเฟรมเดียว เนื่องจากการตรวจจับใบหน้าและการวิเคราะห์อารมณ์ทำได้เฉพาะใบหน้าที่ตรวจจับได้เป็นรายบุคคล

6.3 ข้อเสนอแนะ

1) เพิ่มประสิทธิภาพการทำงาน ปรับปรุงความเร็วในการประมวลผลวิดีโอและลดเวลาในการวิเคราะห์อารมณ์
2) เพิ่มความสามารถในการเล่นคลิปวิดีโอได้นานขึ้นอย่างราบรื่นและมีประสิทธิภาพ
3) เพิ่มความสามารถในการตรวจจับใบหน้าหลายใบหน้าพร้อมกัน
4) เพิ่มความสามารถในการหยุดเล่นวิดีโอชั่วคราวและเล่นต่อจากจุดเดิมได้ ซึ่งอาจช่วยให้การวิเคราะห์อารมณ์ง่ายขึ้น

บรรณานุกรม

[1] A. Team. (2565). "Image Detection, Recognition, And Classification With Machine Learning." [ออนไลน์]. เข้าถึงเมื่อ 10 ตุลาคม 2567. เข้าถึงได้จาก https://azati.ai/image-detection-recognition-and-classification-with-machine-learning/.

[2] c. blog. (2558). "Understanding LSTM Networks." [ออนไลน์]. เข้าถึงเมื่อ 10 ตุลาคม 2567. เข้าถึงได้จาก https://colah.github.io/posts/2015-08-Understanding-LSTMs/.

[3] M. Team. (2566). "Machine Learning คืออะไร ทำไมถึงสำคัญ," [ออนไลน์]. เข้าถึงเมื่อ 10 ตุลาคม 2567. เข้าถึงได้จาก https://blog.mandalasystem.com/th/machine-learning.

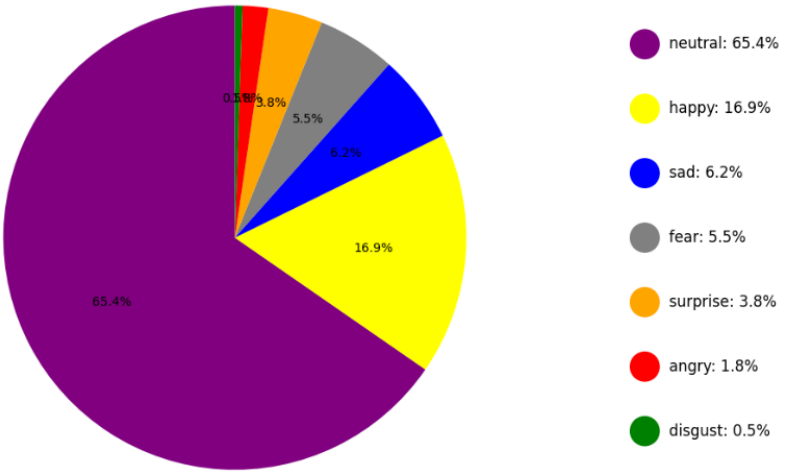
[4] N. Phongchit. (2562). "Convolutional Neural Network (CNN) คืออะไร." [ออนไลน์]. เข้าถึงเมื่อ 10 ตุลาคม 2567. เข้าถึงได้จาก https://medium.com/@natthawatphongchit/%E0%B8%A1%E0%B8%B2%E0%B8%A5%E0%B8%AD%E0%B8%87%E0%B8%94%E0%B8%B9%E0%B8%A7%E0%B8%B4%E0%B8%98%E0%B8%B5%E0%B8%81%E0%B8%B2%E0%B8%A3%E0%B8%84%E0%B8%B4%E0%B8%94%E0%B8%82%E0%B8%AD%E0%B8%87-cnn-%E0%B8%81%E0%B8%B1%E0%B8%99-e3f.

[5] Wikipedia. “Neural network (machine learning).” [ออนไลน์]. เข้าถึงเมื่อ 10 ตุลาคม 2567. เข้าถึงได้จาก https://en.wikipedia.org/wiki/Neural_network_(machine_learning).

[6] S. Sanghkao. (2566). "Image Classification." [ออนไลน์]. เข้าถึงเมื่อ 10 ตุลาคม 2567. เข้าถึงได้จาก https://medium.com/@DodoSirirat/image-classification-108ebbb19514.

ภาคผนวก ก ความหมายของกราฟ

1. สีของอารมณ์



รูปผนวกที่ ก.1 สีของอารมณ์ต่าง ๆ

- **ความโกรธ (Anger) - สีแดง** : แสดงถึงอารมณ์เชิงลบที่มีความเข้มข้นสูง
- **ความขยะแขยง (Disgust) – สีเขียว** : แสดงถึงความรู้สึกไม่พอใจหรือการหลีกเลี่ยง
- **ความกลัว (Fear) – สีเทา** : สื่อถึงความวิตกกังวลหรือความไม่แน่นอน
- **ความสุข (Happy) – สีเหลือง** : แสดงถึงอารมณ์เชิงบวกและความสุข
- **ความเป็นกลาง (Neutral) – สีม่วง** : แสดงถึงความรู้สึกที่ไม่มีอารมณ์ชัดเจน
- **ความเศร้า (Sad) – สีน้ำเงิน** : สื่อถึงความรู้สึกเศร้าหรือท้อแท้
- **ความประหลาดใจ (Surprise) – สีส้ม** : แสดงถึงอารมณ์ที่เกิดจากความคาดไม่ถึง

ภาคผนวก ข การติดตั้งโปรแกรม

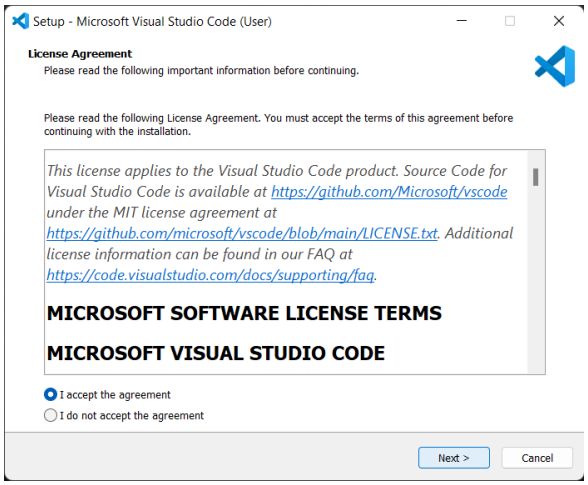
1. วิธีการติดตั้ง Visual Studio Code

- ดาวน์โหลดไฟล์ติดตั้งจากเว็บไซต์ https://code.visualstudio.com/download

- เลือกดาวน์โหลดตามระบบปฏิบัติการตามอุปกรณ์ที่ใช้

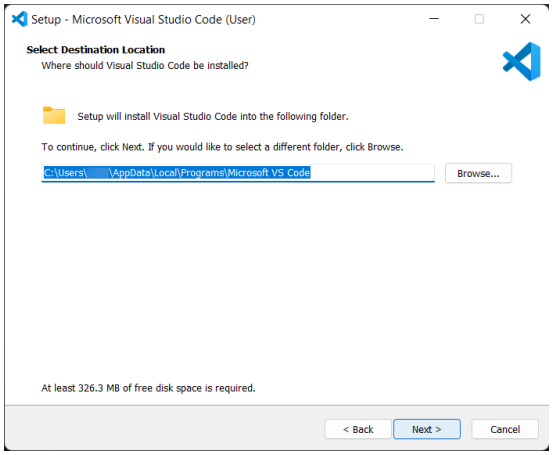
รูปผนวกที่ ข.1 หน้าเว็บไซต์เพื่อดาวน์โหลดไฟล์

- เมื่อดาวน์โหลดเสร็จแล้ว คลิกไฟล์ที่ดาวโหลดและคลิก “Next” เพื่อไปหน้าถัดไป

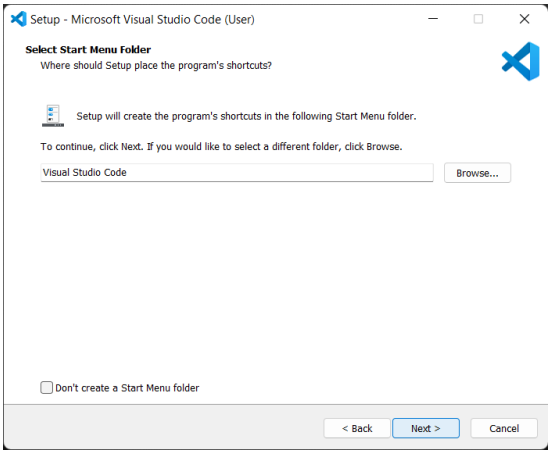


รูปผนวกที่ ข.2 หน้า License Agreement

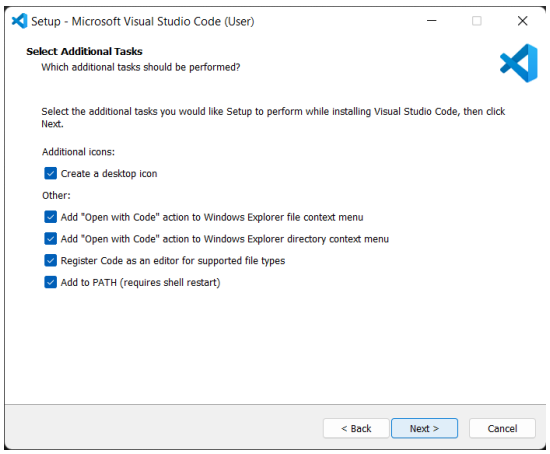
- คลิก “Next”



รูปผนวกที่ ข.3 หน้าเลือกที่อยู่ไฟล์

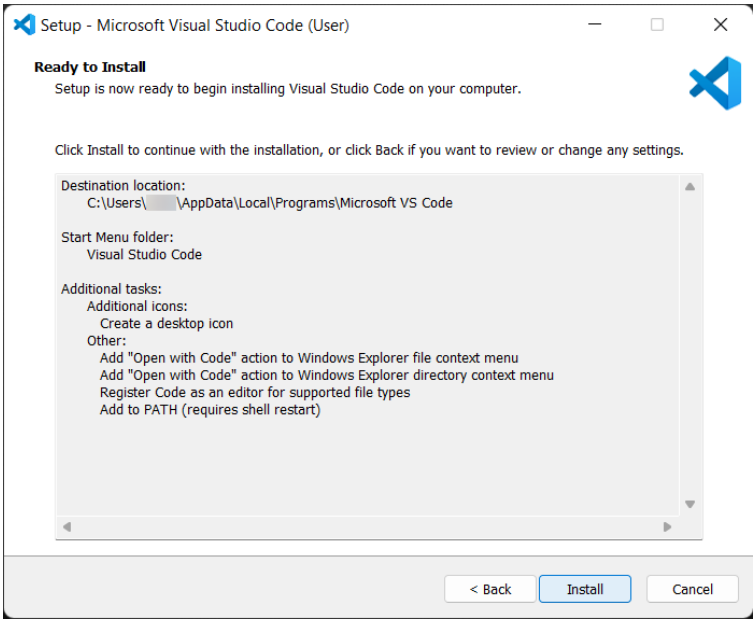


รูปผนวกที่ ข.4 หน้าเลือกโฟลเดอร์เก็บทางลัดโปรแกรมที่จะติดตั้ง

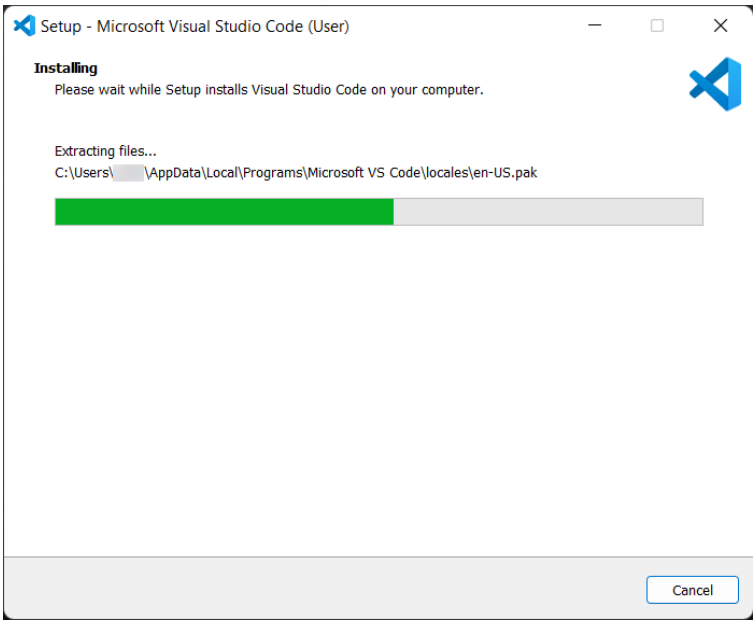


รูปผนวกที่ ข.5 หน้าเลือกการตั้งค่าเพิ่มเติมที่ต้องการในระหว่างการติดตั้งโปรแกรม

- คลิก Install และ รอติดตั้งจนเสร็จสมบูรณ์

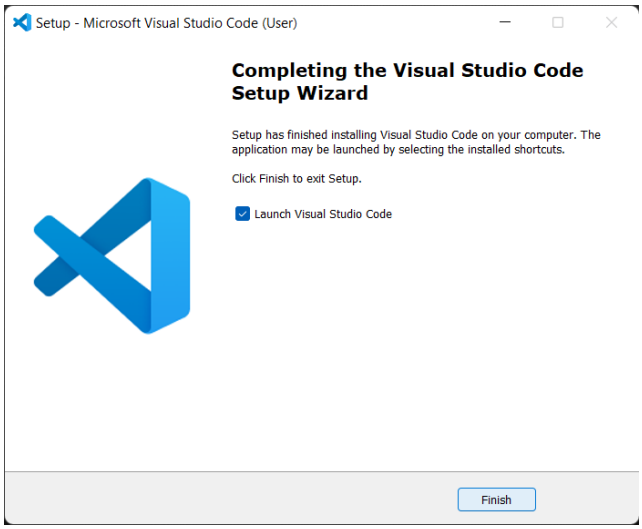


รูปผนวกที่ ข.6 หน้าแสดงรายละเอียดเกี่ยวกับการติดตั้งโปรแกรม

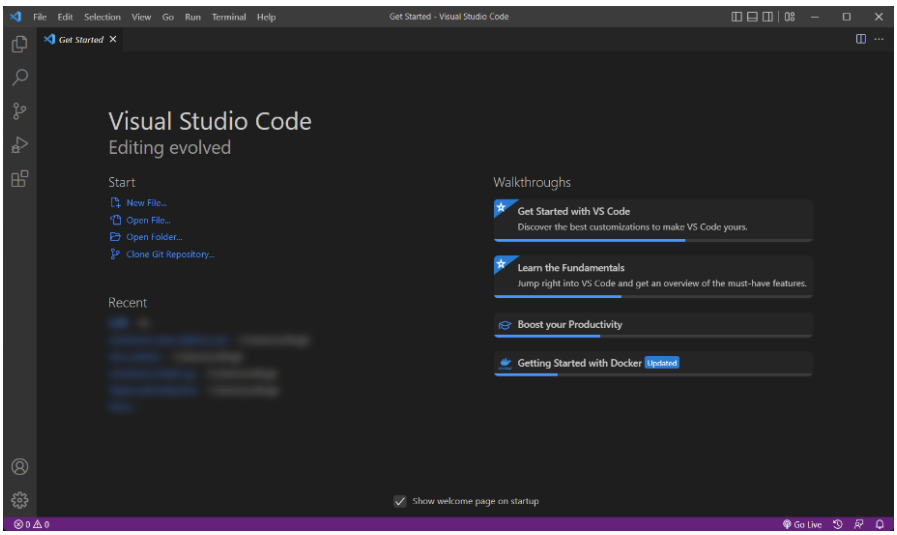


รูปผนวกที่ ข.7 รอดำเนินการจนเสร็จสิ้น

- เมื่อแสดงหน้าจอแบบนี้แล้ว ก็ถือว่าการติดตั้งโปรแกรม Visual Studio Code เสร็จสมบูรณ์



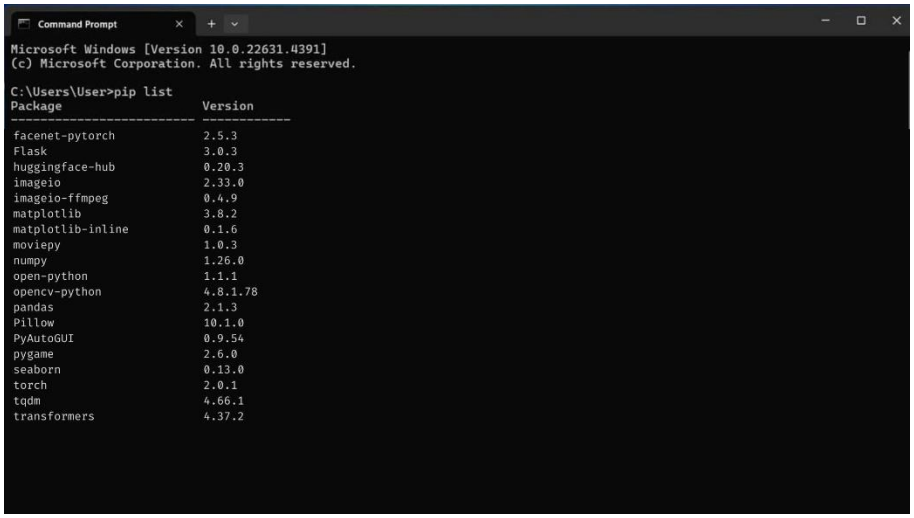
รูปผนวกที่ ข.8 หน้าแสดงสถานะการติดตั้ง



รูปผนวกที่ ข.9 หน้า interface ของ Visual Studio Code

2. การติดตั้งไลบรารี่ ต่าง ๆ ที่ใช้งาน

- pip install opencv-python-headless
- pip install numpy
- pip install torch
- pip install pandas
- pip install matplotlib
- pip install seaborn
- pip install facenet-pytorch
- pip install transformers
- pip install pillow
- pip install moviepy
- pip install tqdm
- pip install pyautogui
- pip install pygame



```
Command Prompt
Microsoft Windows [Version 10.0.22631.4391]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>pip list
Package                 Version
----------------------- ------------
facenet-pytorch         2.5.3
Flask                   3.0.3
huggingface-hub         0.20.3
imageio                 2.33.0
imageio-ffmpeg          0.4.9
matplotlib              3.8.2
matplotlib-inline       0.1.6
moviepy                 1.0.3
numpy                   1.26.0
open-python             1.1.1
opencv-python           4.8.1.78
pandas                  2.1.3
Pillow                  10.1.0
PyAutoGUI               0.9.54
pygame                  2.6.0
seaborn                 0.13.0
torch                   2.0.1
tqdm                    4.66.1
transformers            4.37.2
```

รูปผนวกที่ ข.10 ไลบรารี่ที่ติดตั้ง